

MASTER AZURE – BECOME CLOUD READY · STUDENT NOTES

Storage Account & Azure Data Factory

Session 02 – Live | Student Notes

Blob & ADLS Gen2 · Subscription, Resource Group, Region · OLTP vs OLAP · End-to-end architecture · Azure Data Factory



Introduction – why this session matters

1

Every data project starts with storage

Before data can be cleaned, modelled or reported on, it must land somewhere safe, cheap and scalable. In Azure that place is the Storage Account – usually ADLS Gen2, the data lake.

2

Data must be moved automatically

A company's data lives in many systems (databases, SaaS apps, APIs). Azure Data Factory moves it into the lake on a schedule – no manual copying.

3

Architecture before tools

Knowing OLTP vs OLAP and the end-to-end flow explains WHY each tool exists – and that is exactly what interviews test.

4

Hands-on

Three labs: create an ADLS Gen2 account, create a Blob account and compare them, then create an Azure Data Factory and explore ADF Studio.



Session 01 recap – Cloud Fundamentals

1

What is cloud computing

Renting computing – servers, storage, databases, networking, software – over the internet from a provider such as Azure, instead of owning it.

2

Scalability & autoscaling

Add or remove capacity in minutes; autoscaling does it automatically when demand goes up or down.

3

Geo distribution & agility

Deploy close to users in regions around the world; spin up new services in minutes instead of weeks.

4

Pay-as-you-go

Pay only for what you use – no big up-front hardware purchase (CapEx → OpEx).

5

Cons of cloud & on-prem vs cloud

Watch costs, internet dependency, data-residency/compliance, vendor lock-in. On-prem = you own & run everything.

6

Azure free account

Created last session – it is the subscription we use for today's labs.



Learning objectives

1

Explain

What is inside a Storage Account, and the difference between Blob storage and ADLS Gen2.

2

Organise

Resources with subscription → resource group → resource, and choose a region.

3

Contrast

OLTP and OLAP systems, and describe what an ETL job does.

4

Draw

The end-to-end architecture of an Azure data-engineering project.

5

Create

An ADLS Gen2 account, a Blob account, containers, folders and file uploads.

6

Create & navigate

An Azure Data Factory and the main areas of ADF Studio.



Agenda

#	Topic	Type
1	Azure Storage Account – services & analogy	Concept
2	Subscription, Resource Group & Region	Concept
3	Lab 1 – Create an ADLS Gen2 account	Hands-on
4	OLTP, OLAP & ETL – what a Data Engineer does	Concept
5	End-to-end Data Engineering architecture	Concept
6	Blob storage vs ADLS Gen2	Concept
7	Lab 2 – Create a Blob account & compare	Hands-on
8	Azure Data Factory – concepts	Concept
9	Lab 3 – Create an Azure Data Factory	Hands-on
10	Tour of ADF Studio	Demo
11	Environments (Dev / UAT / Prod) & summary	Concept

Before you start – lab setup

1

Azure access

An Azure subscription (free trial is fine) where you can create resources (Contributor role).

2

Use your own names

Storage and Data Factory names are GLOBALLY unique – add your initials, e.g. stsession02adlsjd.

3

Names used in these notes

rg-session02-lab · stsession02adls · stsession02blob · adf-session02-lab

4

Keep costs low

Choose Standard + LRS in labs. Keep the resource group for the next session; delete it at the end of the course.

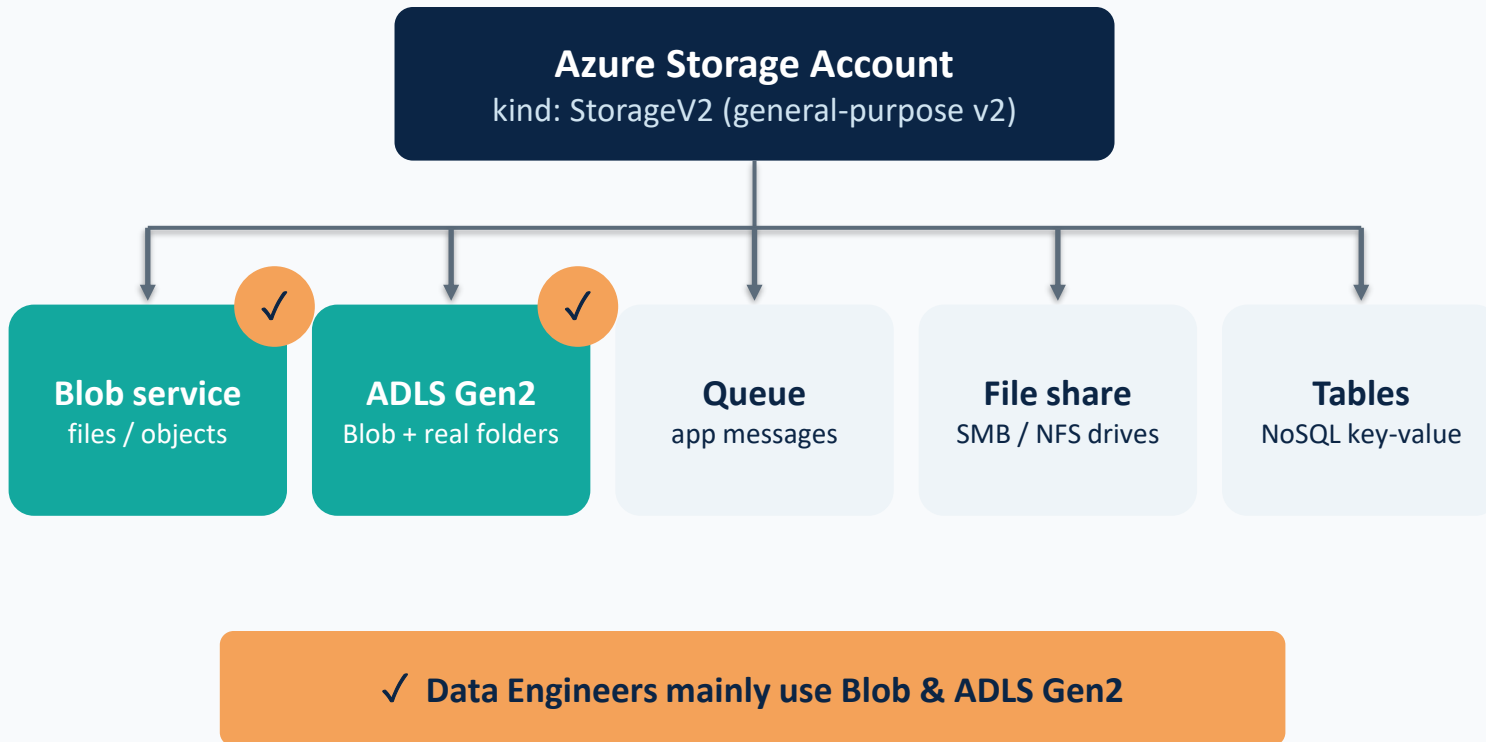
Azure Storage Account

What it is, what's inside, and how it compares to things you already know

01

1

What's inside a Storage Account



Tip: ADLS Gen2 is not a separate resource – it is a StorageV2 account with 'hierarchical namespace' switched on.

KEY POINTS

- One account = several storage services
- Blob service – files/objects of any type
- ADLS Gen2 – Blob + real folders, built for analytics
- Queue, File share, Tables – mostly for app developers
- Data Engineers mainly use Blob & ADLS

1

An analogy you already know



Personal cloud drive

OneDrive · Google Drive ·
Dropbox

≈



Azure Blob / ADLS Gen2

Microsoft Azure

≈



Other clouds

Amazon S3 · Google Cloud
Storage

Same idea: upload any file, organise it, reach it from anywhere

Difference: Azure storage is used by programs & data services at petabyte scale

KEY POINTS

- A personal cloud drive ≈ Azure Blob / ADLS
- Upload any file, reach it from anywhere
- AWS: Amazon S3 · Google Cloud: Cloud Storage
- Difference: used by programs & data services at massive scale

1

Storage center in the Azure portal

> Storage center

Storage center | Blob Storage ... Which storage accounts have been modified recently? +2

fault Directory (contoso.onmicrosoft.com)

Summary Resources

Create Restore Manage view Refresh Export to CSV Open query Group by

Create new storage account

Subscription equals all Resource Group equals all Location equals all Add filter

Name ↑	Type	Kind	Resource Group	Location
stdemo01	Storage account	StorageV2	rg-demo-01	East US
stdemo02	Storage account	StorageV2	rg-demo-05	East US
stdemo03	Storage account	StorageV2	rg-training	East US
stdemo04	Storage account	StorageV2	rg-training	East US

KEY POINTS

- Portal ► Storage center ► Blob Storage ► Resources
- + Create ► 'Create new storage account'
- Kind StorageV2 = general-purpose v2 (the standard)
- Shows Resource Group & Location of each account

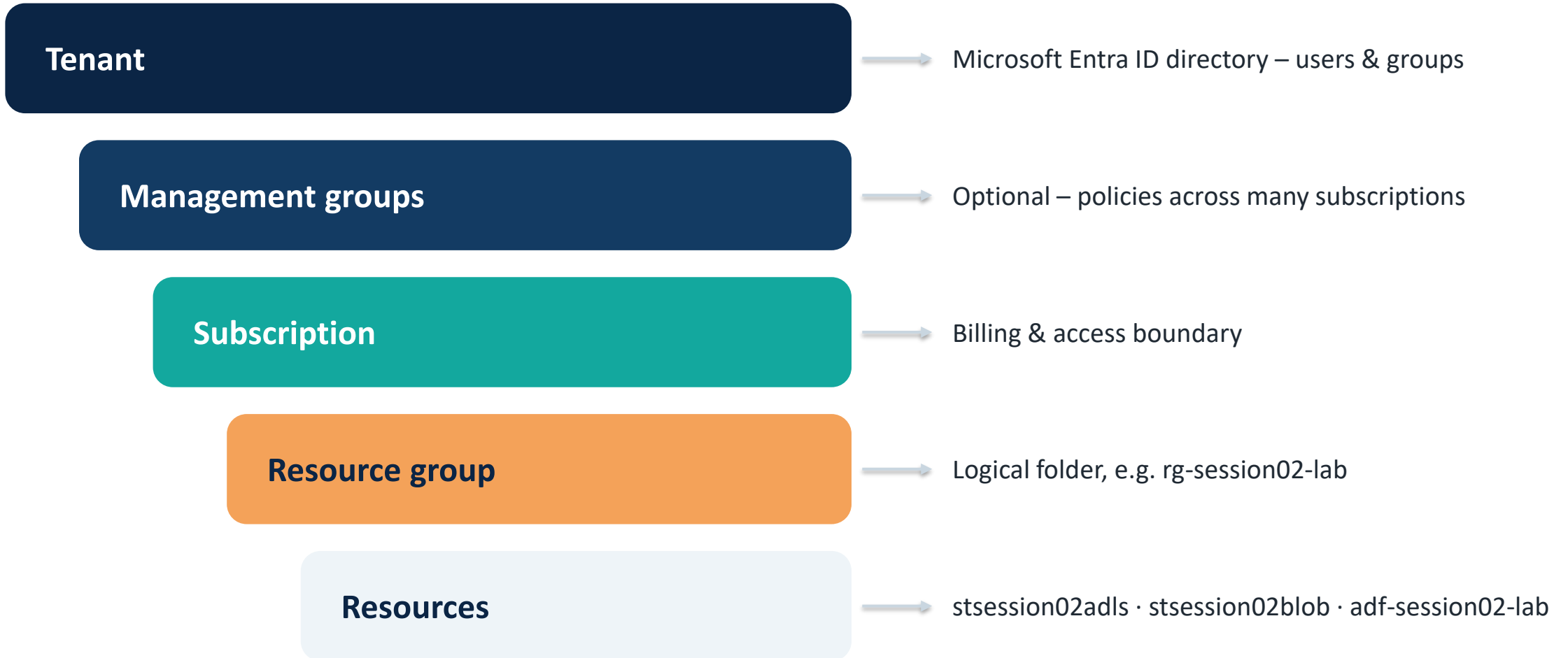
Subscription, Resource Group & Region

How Azure organises, bills and places every resource

02

2

The Azure resource hierarchy



Permissions (IAM) & policies flow DOWN the hierarchy. Region = where each resource physically lives.

2 Subscription – the billing boundary

The screenshot shows the 'Create a storage account' page in the Microsoft Azure portal. The page is titled 'Create a storage account' and includes a brief description of Azure Storage. The 'Project details' section is active, showing a form to select a subscription and a resource group. The 'Subscription' dropdown is set to 'Visual Studio Enterprise Subscription' and the 'Resource group' dropdown is set to 'rg-training'. Below the resource group dropdown is a link to 'Create new'. The 'Instance details' section is partially visible at the bottom, showing a text input for 'Storage account name *'. At the bottom of the form are three buttons: 'Previous', 'Next', and 'Review + create'. A 'Give feedback' link is also present in the bottom right corner.

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home > Storage center | Blob Storage

Create a storage account

Azure Storage is a Microsoft-managed service providing cloud storage that is highly available, secure, durable, scalable, and redundant. Azure Storage includes Azure Blobs (objects), Azure Data Lake Storage Gen2, Azure Files, Azure Queues, and Azure Tables. The cost of your storage account depends on the usage and the options you choose below. [Learn more about Azure storage accounts](#)

Project details

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * Visual Studio Enterprise Subscription

Resource group * rg-training

[Create new](#)

Instance details

Storage account name *

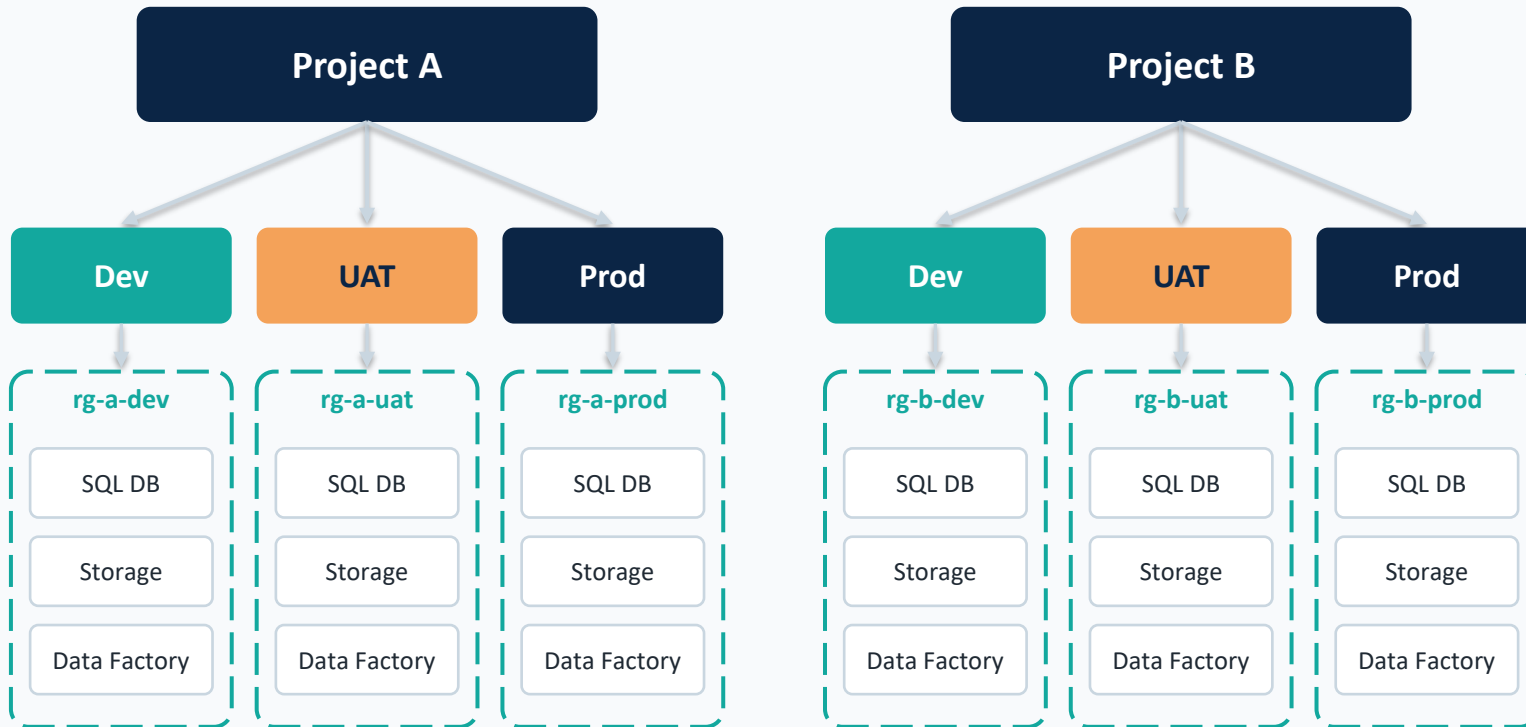
Previous Next Review + create

[Give feedback](#)

KEY POINTS

- First field in every 'Create' form
- Answers: who pays, which access rules apply
- Example: Visual Studio Enterprise Subscription
- Companies often separate Prod / Non-prod subscriptions

Why resource groups exist



One resource group per project per environment → easy to secure, cost & clean up

KEY POINTS

- Companies run many projects
- Each project has Dev, UAT and Prod environments
- Each environment has its own resources
- A resource group keeps each set together
- e.g. rg-a-dev, rg-a-uat, rg-a-prod

A real resource group

The screenshot displays the Microsoft Azure portal interface. At the top, the header includes the Microsoft Azure logo, a search bar, and a Copilot button. The breadcrumb navigation shows 'Home > Storage center | Blob Storage'. The main heading is 'rg-project-demo', with a sub-label 'Resource group'. Below this, there are two buttons: 'How to manage changes with deployment tools?' and 'Generate Bicep code to duplicate this resource group.' with a '+1' button. A search bar is also present. The left sidebar contains a navigation menu with options: Overview (selected), Activity log, Access control (IAM), Tags, Resource visualizer, Events, Settings, Cost Management, Monitoring, Automation, and Help. The main content area shows a table of resources within the 'rg-project-demo' group. The table has columns for checkboxes, resource names, resource types, and locations. The resources listed are:

Resource Name	Resource Type	Location
Failure Anomalies - logic-demo	Smart detector alert rule	Global
Failure Anomalies - func-demo	Smart detector alert rule	Global
office365	API Connection	Central US
outlook	API Connection	Central US
outlook-2	API Connection	Central US
stdemo05	Storage account	Central US
sql-demo	SQL server	East US
func-demo	Function App	Central US
func-demo	Application Insights	Central US
kv-demo	Key vault	East US
stdemo06	Storage account	Central US

KEY POINTS

- Holds MANY resource types together
- Resources can be in DIFFERENT regions
- Every resource lives in exactly ONE resource group
- IAM, Tags, Cost Management at group level
- Delete the group $\Rightarrow$ everything inside is deleted

Create a new resource group

Home > Storage center | Blob Storage

Create a storage account

Resource group * rg-training [Create new](#)

Instance details

Storage account name *

Region *

Preferred storage type

Performance * ☒ Standard: Recommended for most scenarios (general-purpose v2 account)

Microsoft Azure Search resources, services, and docs (G+/) Copilot

Home > Storage center | Blob Storage

Create a storage account

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * Visual Studio Enterprise Subscription

Resource group * (New) rg-session02-lab [Create new](#)

Instance details

Storage account name *

Region * (US) East US [Deploy to an Azure Extended Zone](#)

KEY POINTS

- Click 'Create new' under Resource group
- 'A container that holds related resources for an Azure solution'
- Name: rg-session02-lab
- '(New)' = created together with the storage account

2 Region – where your data physically lives

Americas

- East US
- West US
- Canada Central
- Brazil South

Paired regions

hold geo-redundant copies

South India ⇄ Central India

Europe

- UK South
- West Europe
- North Europe

Africa & Middle East

- South Africa North
- UAE North

East US ⇄ West US

Asia Pacific

- South India
- Central India
- Japan East
- Australia East

Pick by: latency · compliance · cost · service availability

UK South ⇄ UK West

Keep services that work together in the SAME region

KEY POINTS

- Region = geographic area with Azure data centres
- Latency: close to users & services
- Compliance: data-residency laws
- Cost & feature availability differ
- Most regions have a PAIRED region

Instance details – name & region

Home > Storage center | Blob Storage

Create a storage account ...

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription *

Resource group *
[Create new](#)

Instance details

Storage account name * ⓘ

Region * ⓘ
[Deploy to an Azure Extended Zone](#)

Preferred storage type

[Previous](#) [Next](#) [Review + create](#)

KEY POINTS

- Storage account name: stsession02adls
- 3–24 chars, lowercase letters & numbers only
- Globally unique – it becomes a URL
- Region selected: (Asia Pacific) South India

Lab 1 – Create an ADLS Gen2 Account

Performance, redundancy, and the one checkbox that makes it a Data Lake

03

Performance & redundancy

Home > Storage center | Blob Storage

Create a storage account ...

Deploy to an Azure Extended Zone

Preferred storage type: Choose preferred storage type

Performance * ⓘ

☒ **Standard:** Recommended for most scenarios (general-purpose v2 account)

☐ **Premium:** Recommended for scenarios that require low latency.

Redundancy * ⓘ

Geo-redundant storage (GRS)

☒ Make read access to data available in the event of regional unavailability.

☐ Geo priority replication guarantees Blob storage data is geo-replicated within 15 minutes.

Previous Next Review + create

KEY POINTS

- Performance: Standard (general-purpose v2)
- Premium = low latency, higher cost
- GRS + 'read access if region unavailable' = RA-GRS
- Labs: LRS is cheapest and fine

3 Advanced tab – hierarchical namespace

Home > Storage center | Blob Storage

Create a storage account ...

Hierarchical Namespace

Hierarchical namespace, complemented by Data Lake Storage Gen2 endpoint, enables file and directory semantics, accelerates big data analytics workloads, and enables access control lists (ACLs) [Learn more](#)

Enable hierarchical namespace ☒ ⓘ

Access protocols

Blob and Data Lake Gen2 endpoints are provisioned by default [Learn more](#)

Enable SFTP ☐ ⓘ

Enable network file system v3 ☐ ⓘ

Blob storage

[Previous](#) [Next](#) [Review + create](#)

ⓘ The current combination of storage account kind, performance, replication, and location does not support the NFS v3 feature. [Learn more about NFS v3](#)

KEY POINTS

- 'Enables file and directory semantics'
- 'Accelerates big data analytics workloads'
- 'Enables access control lists (ACLs)'
- SFTP / NFS v3 options live here too

✓ Enable hierarchical namespace

Home > Storage center | Blob Storage

Create a storage account ...

Minimum TLS version ⓘ Version 1.2 ▼

Permitted scope for copy operations (preview) ⓘ From any storage account ▼

Hierarchical Namespace

Hierarchical namespace, complemented by Data Lake Storage Gen2 endpoint, enables file and directory semantics, accelerates big data analytics workloads, and enables access control lists (ACLs) [Learn more](#) ↗

Enable hierarchical namespace ⓘ ☒

Access protocols

Blob and Data Lake Gen2 endpoints are provisioned by default [Learn more](#) ↗

Enable SFTP ⓘ ☐

Enable network file system v3 ⓘ ☐

Previous Next Review + create

KEY POINTS

- THIS checkbox turns the account into ADLS Gen2
- Same account kind (StorageV2) – not a new service
- Can be enabled later on Blob; can't be switched off
- Minimum TLS 1.2 (secure default)

Wizard tabs & Tags

Create a storage account ...

Basics Advanced Networking Data protection Encryption Tags Review + create

Name	Value	Resource
	:	

Previous Next Review + create

KEY POINTS

- Basics ▶ Advanced ▶ Networking ▶ Data protection ▶ Encryption ▶ Tags ▶ Review + create
- Defaults are fine for this lab
- Tags = Name:Value labels (env=dev, owner=you)
- Tags drive cost reports & governance

Create a storage account ...

[View automation template](#)

Basics

Subscription	Visual Studio Enterprise Subscription
Resource group	rg-session02-lab
Location	South India
Storage account name	stsession02adls
Preferred storage type	
Performance	Standard
Replication	Read-access geo-redundant storage (RA-GRS)

Advanced

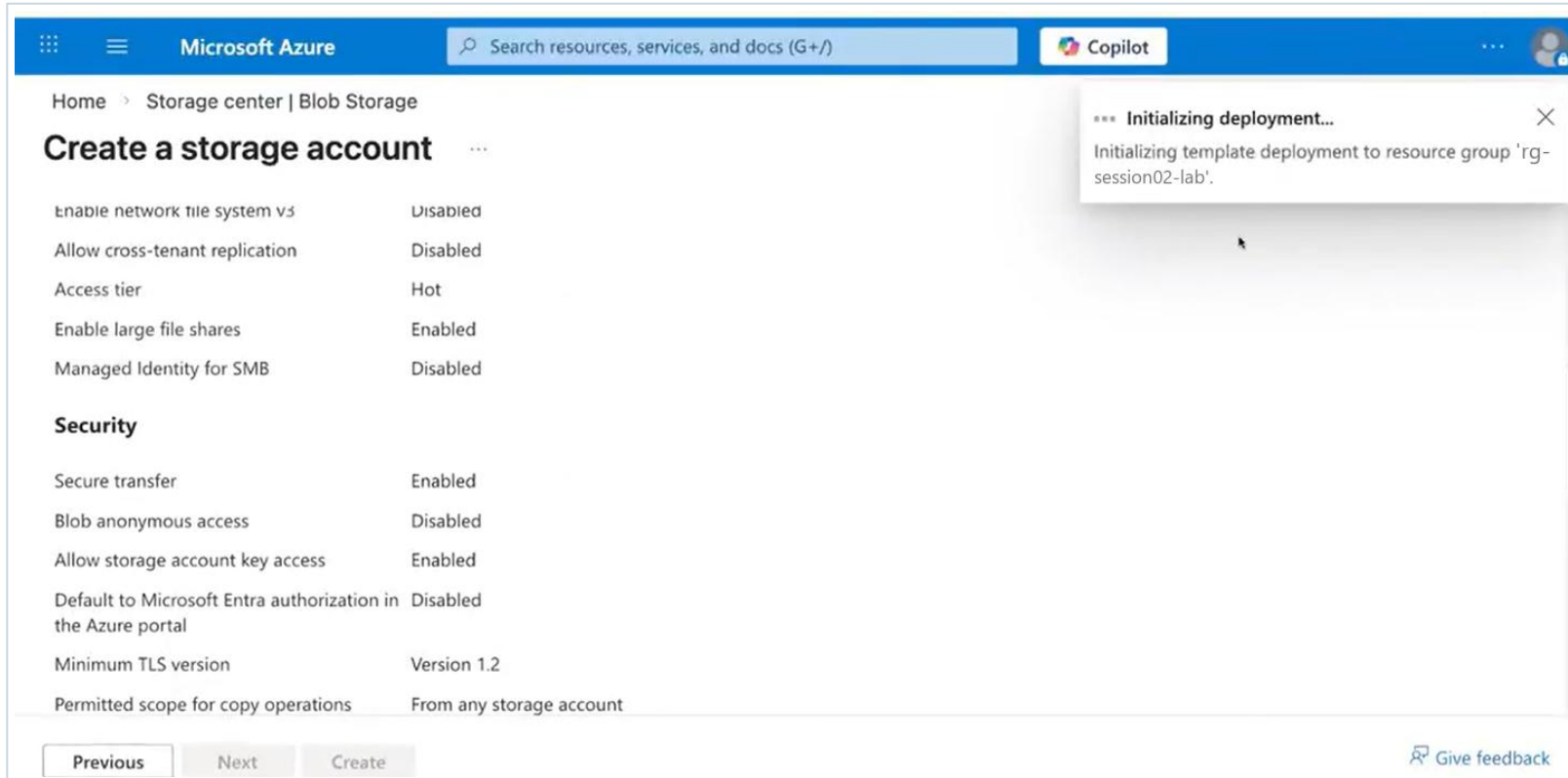
Enable hierarchical namespace	Enabled
-------------------------------	---------

[Previous](#) [Next](#) [Create](#)

KEY POINTS

- Always read the summary before 'Create'
- Resource group rg-session02-lab · South India
- Standard · RA-GRS
- Enable hierarchical namespace: Enabled ✓
- 'View automation template' = ARM template (IaC)

Security defaults & deployment



Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home > Storage center | Blob Storage

Create a storage account

Enable network file system v3	Disabled
Allow cross-tenant replication	Disabled
Access tier	Hot
Enable large file shares	Enabled
Managed Identity for SMB	Disabled

Security

Secure transfer	Enabled
Blob anonymous access	Disabled
Allow storage account key access	Enabled
Default to Microsoft Entra authorization in the Azure portal	Disabled
Minimum TLS version	Version 1.2
Permitted scope for copy operations	From any storage account

Previous Next Create

Give feedback

*** Initializing deployment...
Initializing template deployment to resource group 'rg-session02-lab'.

KEY POINTS

- Access tier: Hot (frequently used data)
- Secure transfer (HTTPS only): Enabled
- Blob anonymous access: Disabled
- Min TLS 1.2
- Create ⇒ 'Initializing template deployment...'

Deployment succeeded

The screenshot shows the Microsoft Azure portal interface. At the top, there's a navigation bar with the Microsoft Azure logo, a search bar, and a Copilot button. Below the navigation bar, the main content area displays the 'Overview' page for a deployment named 'stsession02adls_deploy'. The deployment is marked as 'Deployment succeeded' with a green checkmark. A notification banner at the top right states: 'Deployment succeeded. Deployment 'stsession02adls_deploy' resource group 'rg-session02-lab' was successful.' Below this, the main heading says 'Your deployment is complete'. The deployment details include: Deployment name: stsession02..., Subscription: Visual Studio Ente..., Resource group: rg-session02-..., Start time: 2/3/2026, 8:30:49 AM, and Correlation ID: 00000000-0000-0000-0000-00000000. There are buttons for 'Go to resource' and 'Pin to dashboard'. On the left sidebar, there are links for 'Overview', 'Inputs', 'Outputs', and 'Template'. At the bottom, there's a 'Give feedback' section with a link to 'Tell us about your experience with deployment'. On the right side of the main content area, there are two promotional cards: 'Cost Management' and 'Microsoft Defender for Cloud'.

KEY POINTS

- Every Create = a Deployment record
- Deployment name, start time, correlation ID
- Go to resource / Pin to dashboard
- Tip: set up Cost Management alerts

Storage account overview

The screenshot displays the Azure portal interface for a storage account. The top navigation bar includes the Microsoft Azure logo, a search bar, and the Copilot icon. The breadcrumb trail shows 'Home > stsession02adls_deploy'. The main header area shows the storage account name 'stsession02adls' and a 'Storage account' link. Below this is a search bar and a set of action buttons: Upload, Open in Explorer, Delete, Move, Refresh, Open in mobile, CLI / PS, and Feedback. The left sidebar contains a list of navigation options: Overview (selected), Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, and Data storage. The main content area is titled 'Overview' and features a 'JSON View' link. It displays a table of essential information about the storage account:

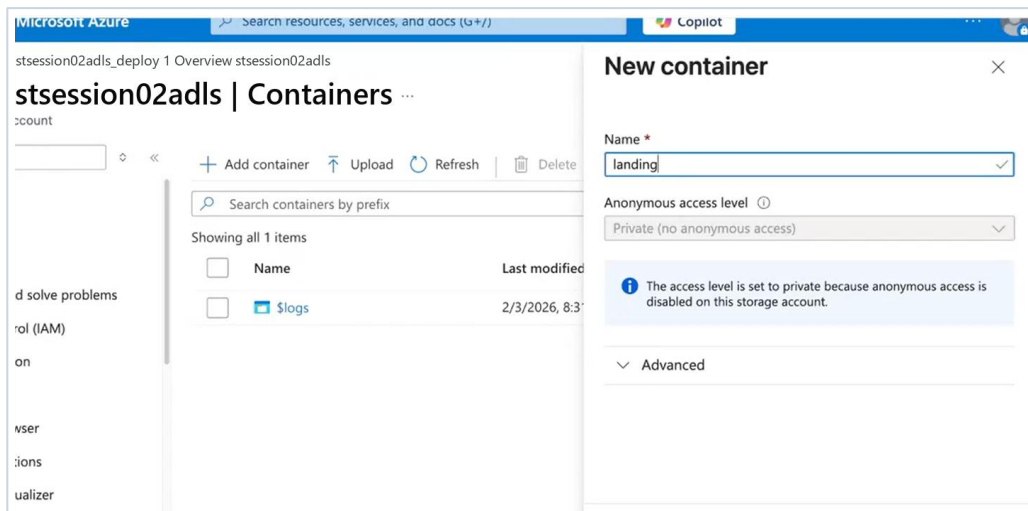
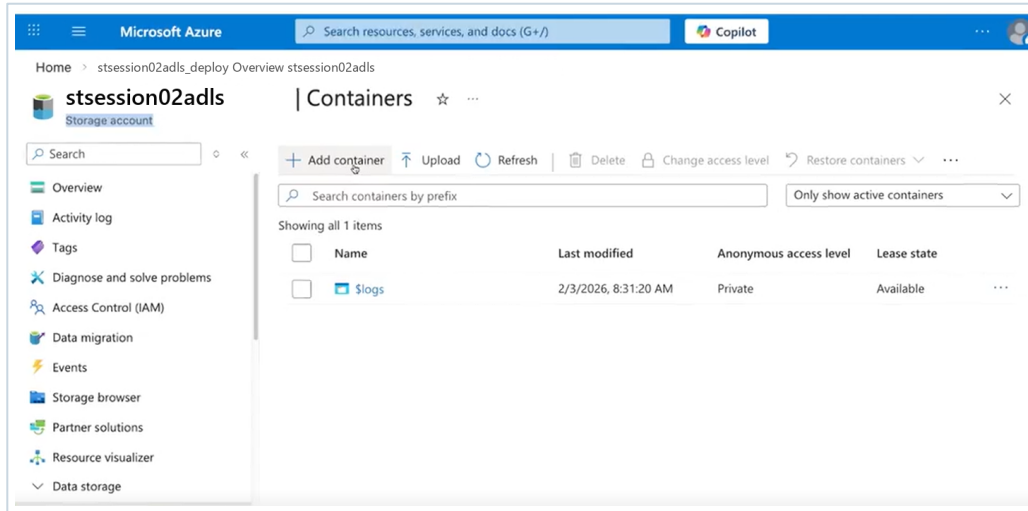
Essentials	
Resource group (move)	Performance
rg-session02-lab	Standard
Location	Replication
southindia	Read-access geo-redundant storage (RA-GRS)
Primary/Secondary Location	Account kind
Primary: South India, Secondary: Central India	StorageV2 (general purpose v2)
Subscription (move)	Provisioning state
Visual Studio Enterprise Subscription	Succeeded
Subscription ID	Created
00000000-0000-0000-0000-000000000000	2/3/2026, 8:30:52 AM
Disk state	
Primary: Available, Secondary: Available	

KEY POINTS

- Resource group, Location, Subscription – the 3 concepts
- Primary: South India · Secondary: Central India (RA-GRS)
- Account kind: StorageV2 (general purpose v2)
- Disk state: Primary & Secondary Available

3

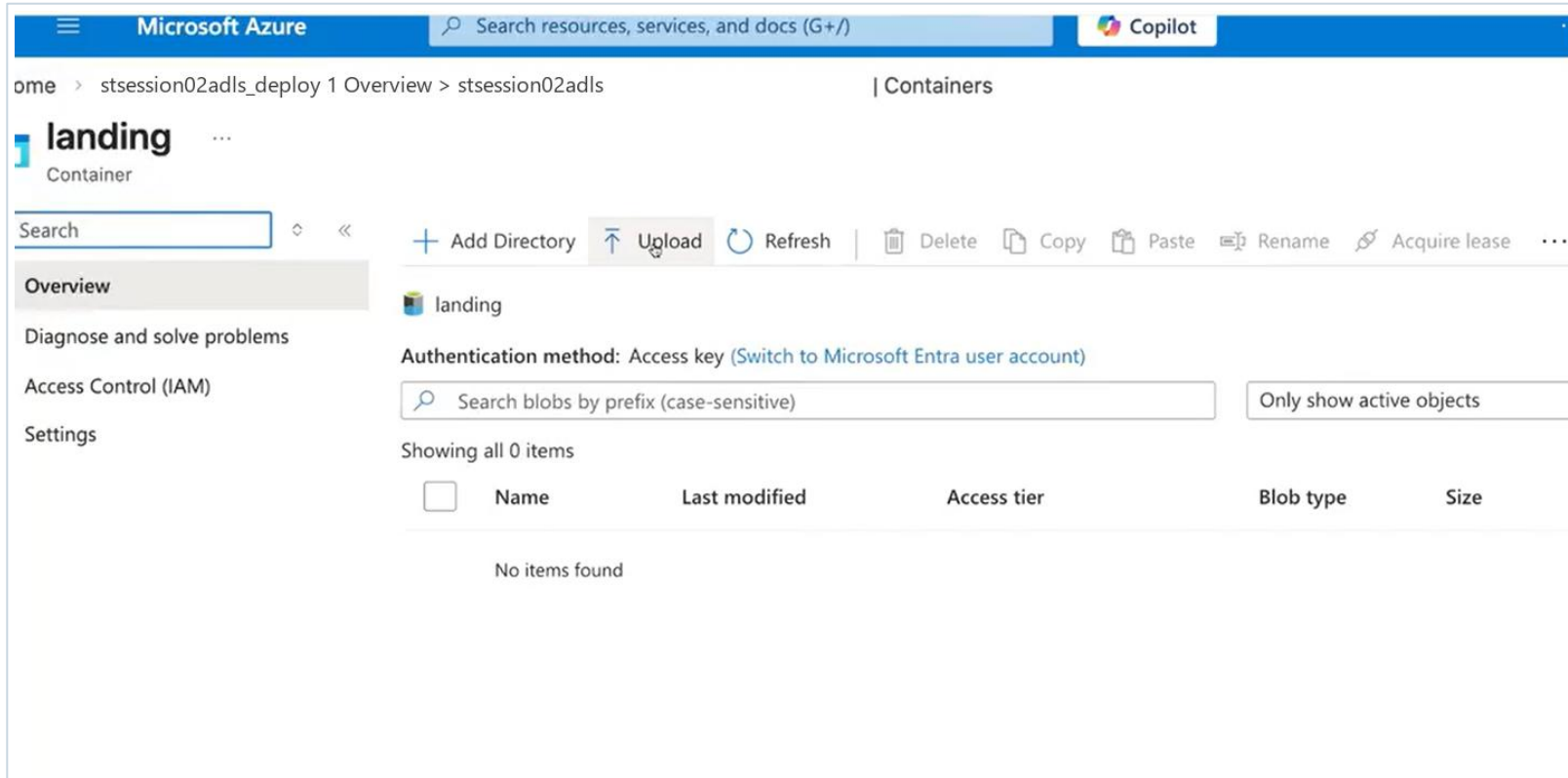
Containers – create 'landing'



KEY POINTS

- Container = top-level folder (ADLS: 'file system')
- \$logs is a system container
- Create 'landing' – the raw landing zone
- Access level locked to Private

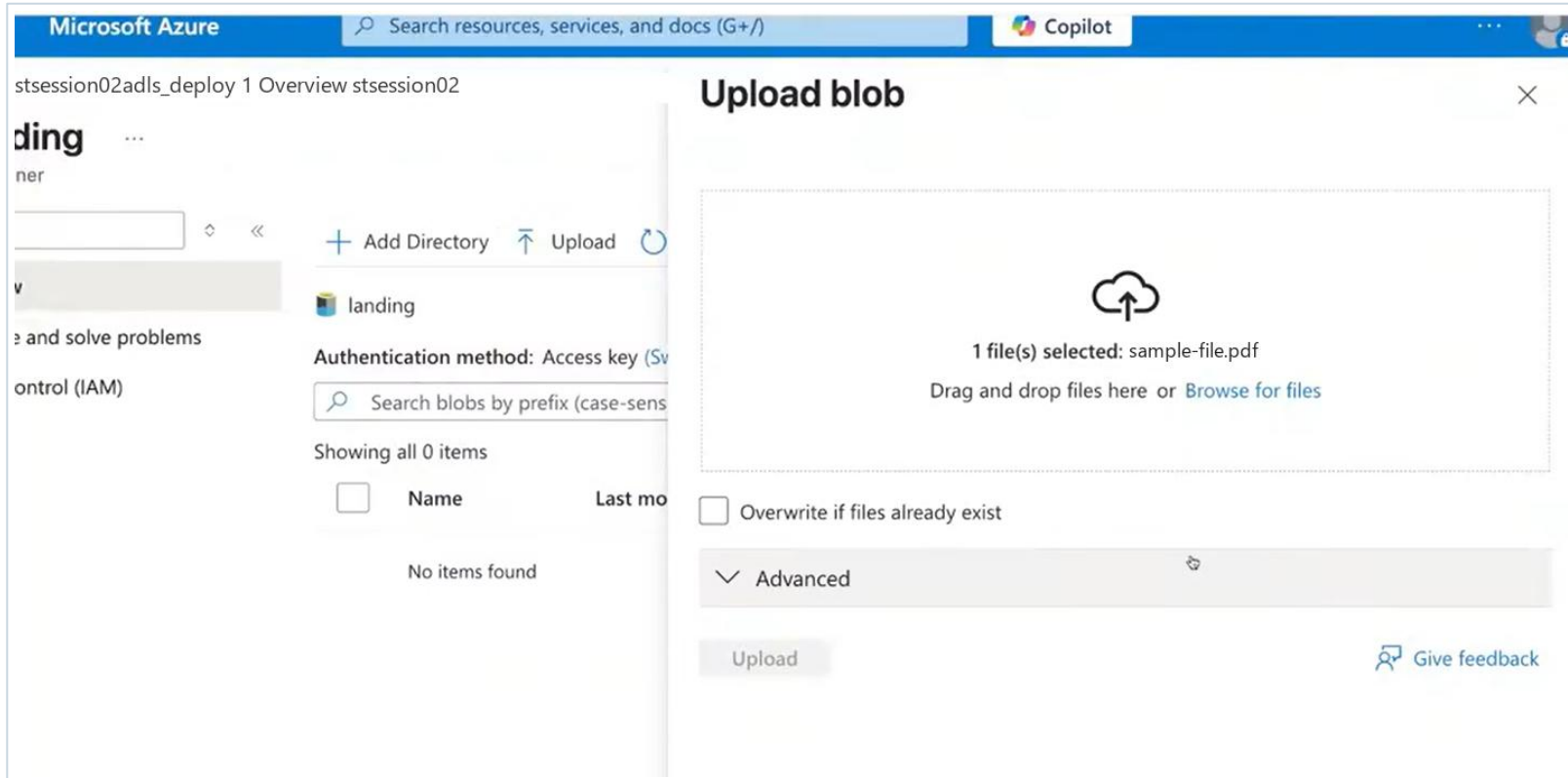
Inside the container (ADLS)



KEY POINTS

- '+ Add Directory' = a REAL directory (HNS on)
- Upload, Rename, Acquire lease ...
- Auth method: Access key (or Entra ID user)
- Production: Entra ID + RBAC / ACLs

Upload a file



KEY POINTS

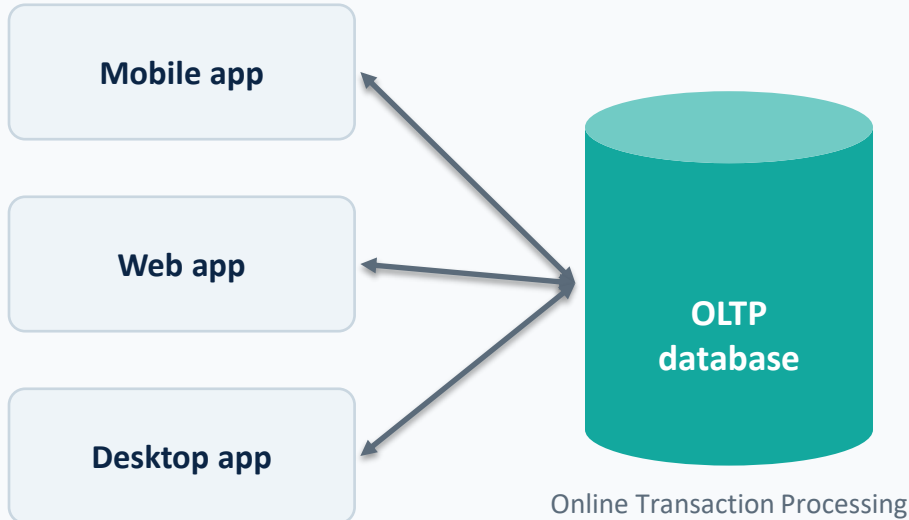
- Upload ► drag & drop or Browse for files
- Any file type: CSV, JSON, Parquet, PDF ...
- 'Overwrite if files already exist'
- Advanced: blob type, block size, tier, folder

OLTP, OLAP & ETL

What a Data Engineer actually does – and why

04

OLTP – the application database



Built for:

- Many small, fast transactions
- Read fast · write fast
- Small data per operation
- Many users at once
- Normalised tables

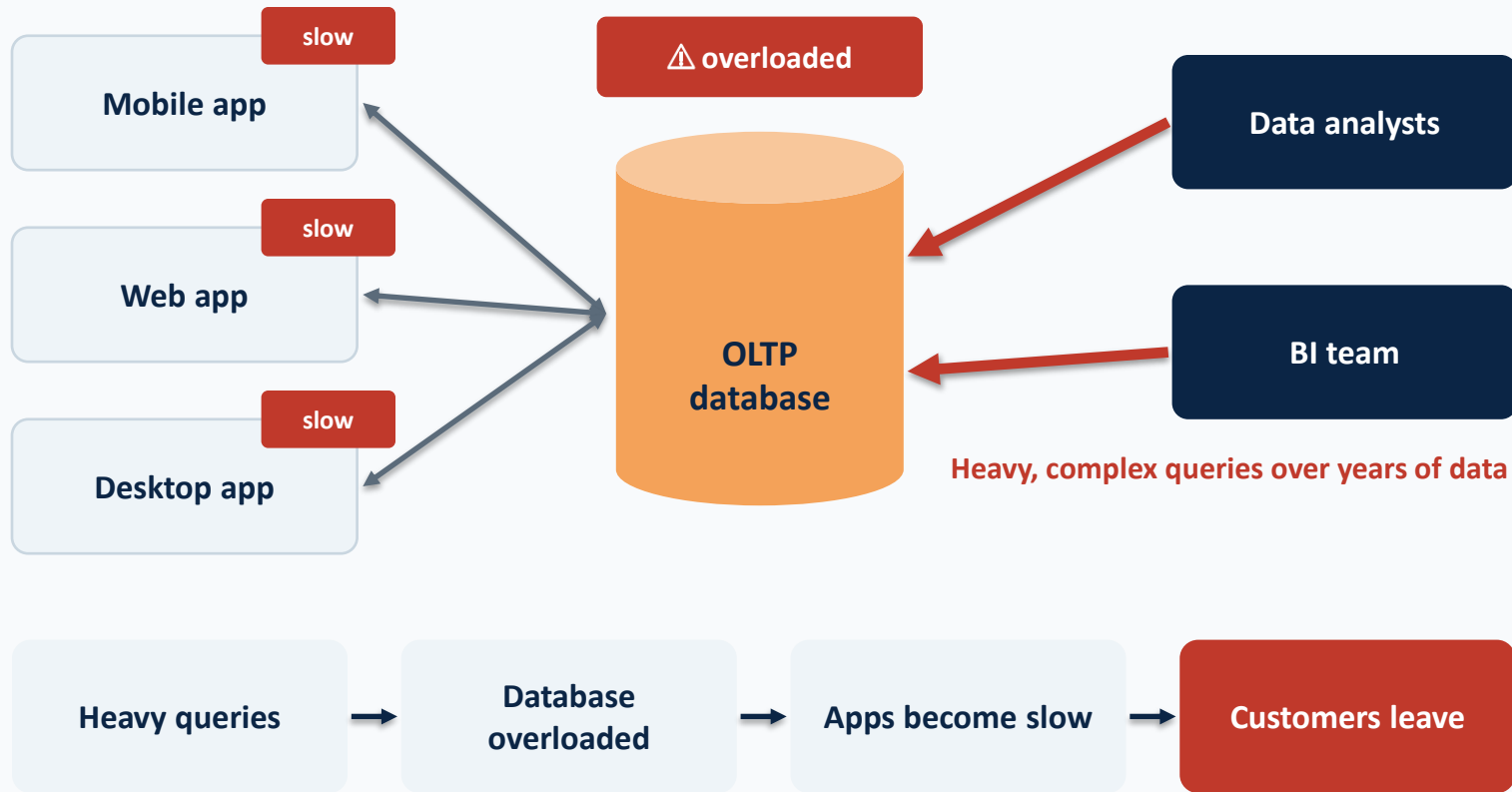
Operations: place an order · update an address · show one basket

Typical OLTP databases: Oracle · MySQL · SQL Server · PostgreSQL · Azure SQL

KEY POINTS

- Mobile, web & desktop apps ⇒ one database
- OLTP = Online Transaction Processing
- Read fast, write fast ...
- ... but only SMALL amounts of data per operation

The problem: analytics on OLTP

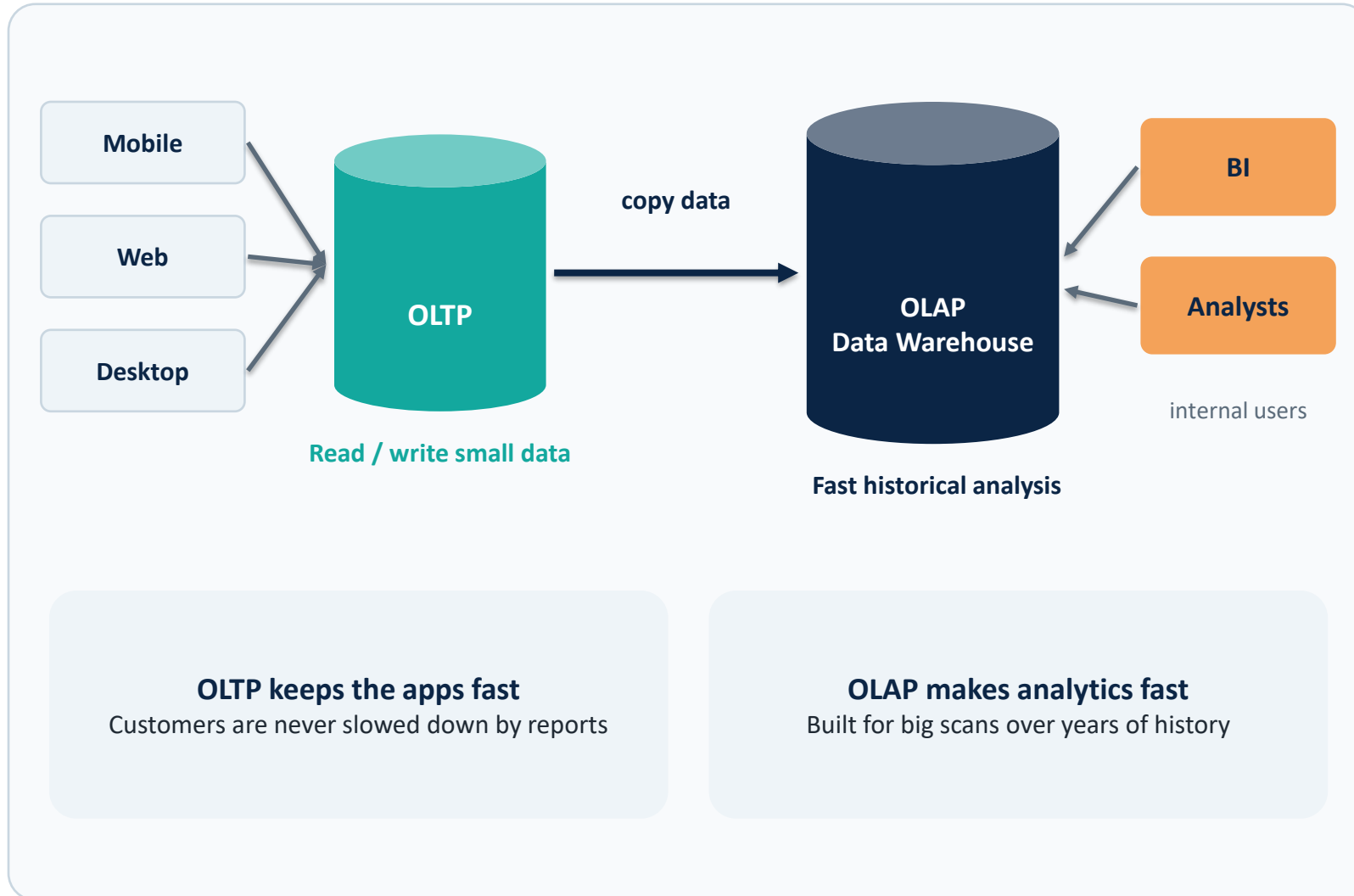


Golden rule: never run heavy analytics on the production OLTP database.

KEY POINTS

- Analysts & BI teams query the same DB
- Complex queries over months/years of data
- Heavy queries $\Rightarrow$ database overloaded
- Slow apps for customers $\Rightarrow$ business impact

4 The solution: a separate OLAP system



KEY POINTS

- Copy data OLTP $\Rightarrow$ OLAP (Online Analytical Processing)
- OLAP = Data Warehouse
- BI & analysts (internal users) query OLAP
- OLTP: R/W small data · OLAP: fast historical analysis

4 ETL – the bridge (the Data Engineer's job)



OLTP sources
Oracle · MySQL · SQL Server · PostgreSQL

OLAP targets
Synapse · Fabric Warehouse · Oracle DW · Db2 Warehouse

Building & running these ETL jobs = the Data Engineer's core job

KEY POINTS

- ETL jobs move data OLTP $\Rightarrow$ OLAP
- E = Extract · T = Transform · L = Load
- Modern variant: ELT (load first, transform in the lake)
- Building ETL = the Data Engineer's core job

OLTP vs OLAP

Aspect	OLTP	OLAP
Full name	Online Transaction Processing	Online Analytical Processing
Purpose	Run the business	Analyse the business
Users	Customers & applications	Analysts, BI, data scientists
Operations	Many small INSERT / UPDATE / point reads	Few large, complex SELECTs
Data volume / query	Small (rows)	Huge (millions+ rows)
Data age	Current	Historical (years)
Design	Normalised (3NF)	Denormalised (star schema)
Examples	Oracle, MySQL, SQL Server, PostgreSQL	Synapse, Fabric, Oracle DW, Db2 Warehouse

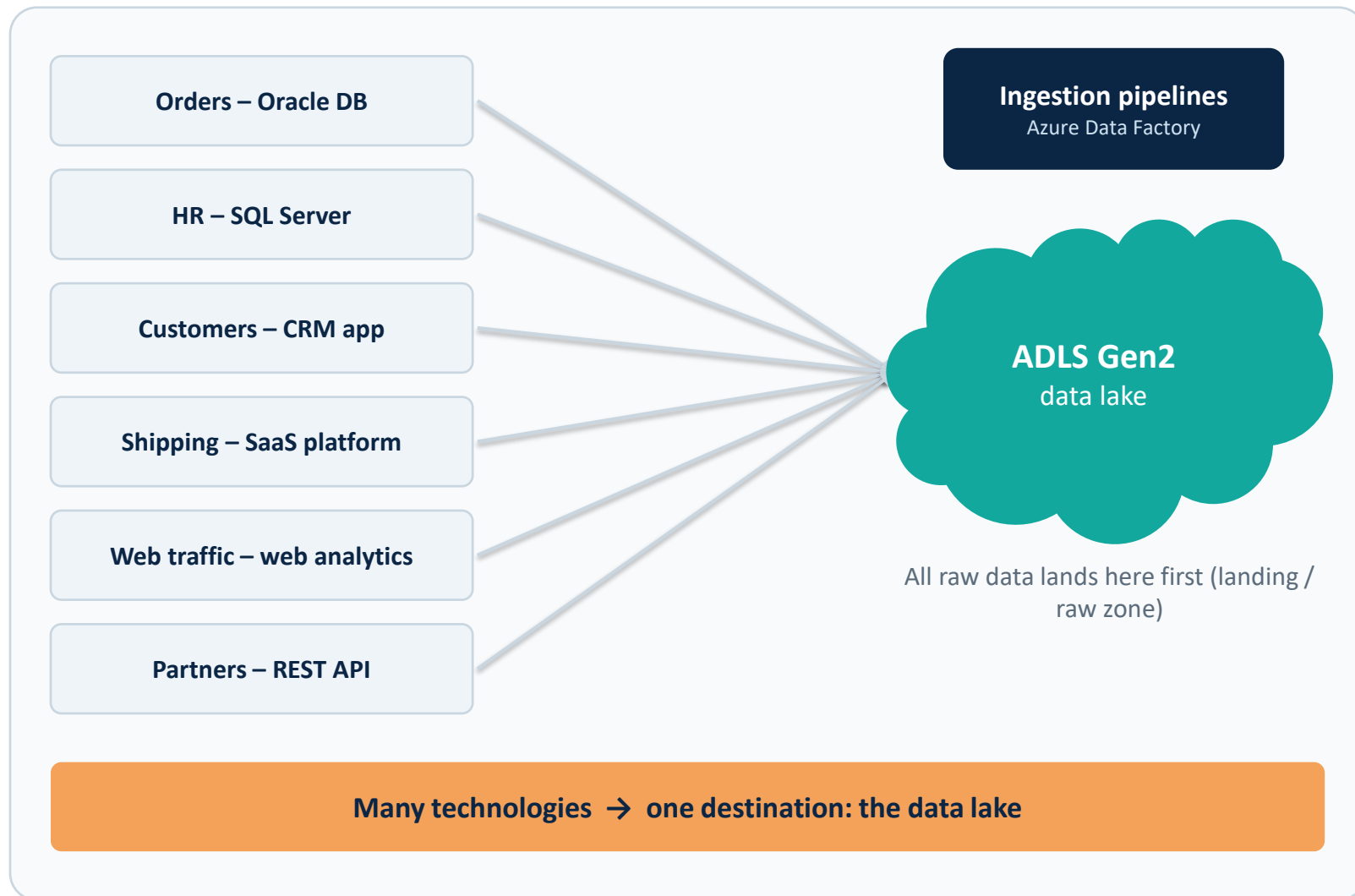
End-to-End Data Engineering Architecture

From many source systems to BI, analytics and AI



05

Many sources $\Rightarrow$ ingestion $\Rightarrow$ ADLS

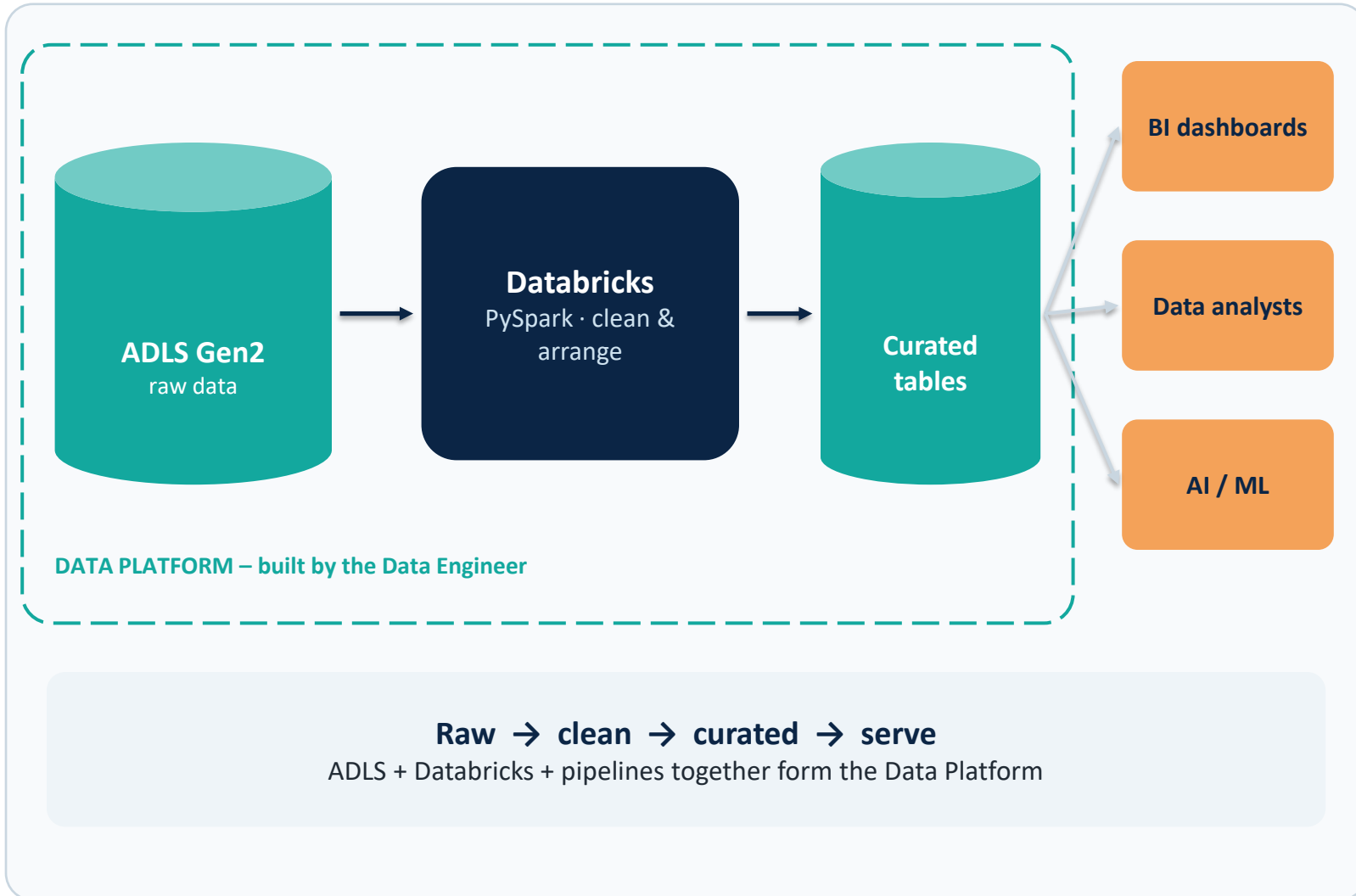


KEY POINTS

- Real companies have many different sources
- Databases, SaaS apps, analytics tools, APIs
- Ingestion pipelines copy everything to one place
- That place is the data lake: ADLS Gen2

5

Clean & arrange $\Rightarrow$ serve

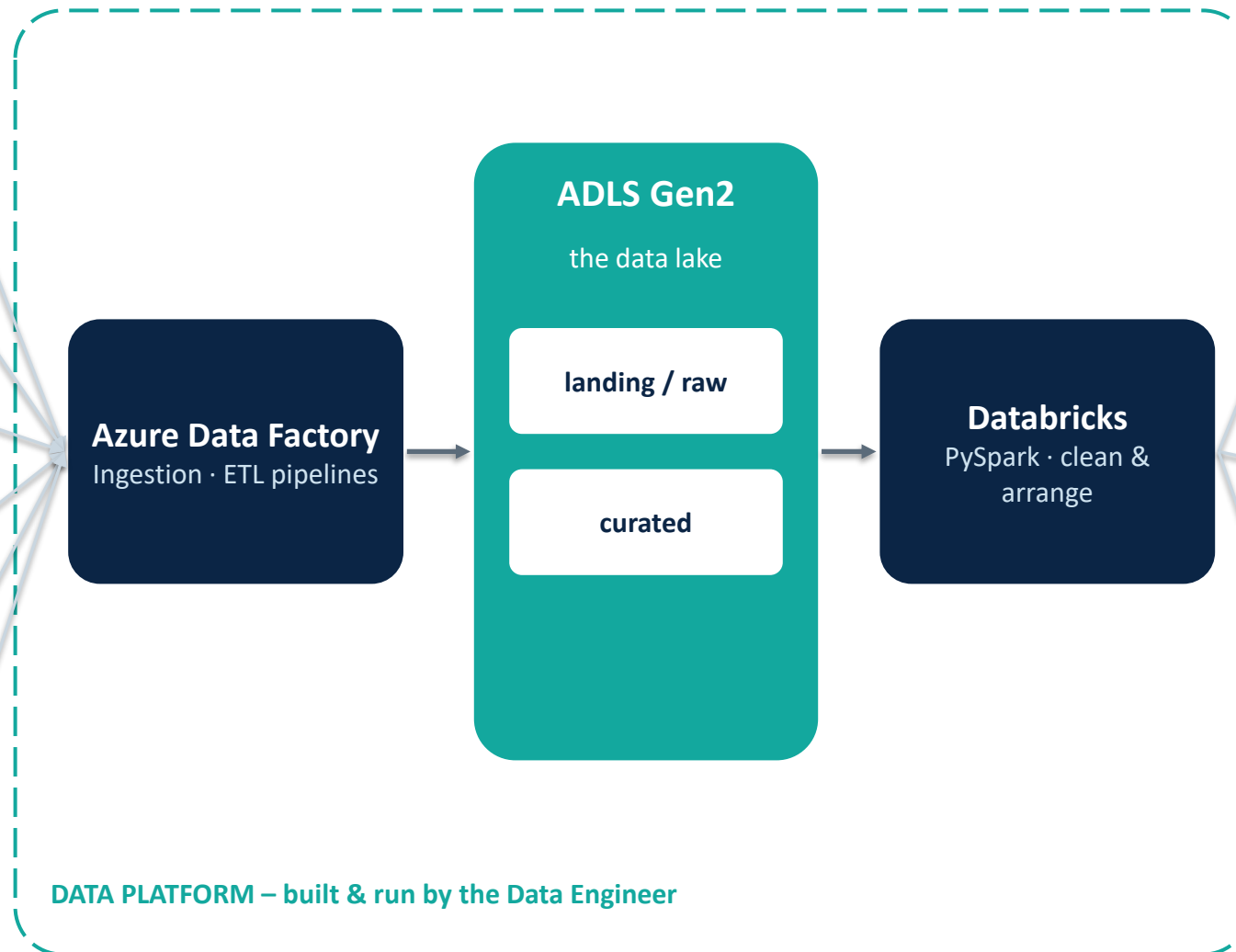


KEY POINTS

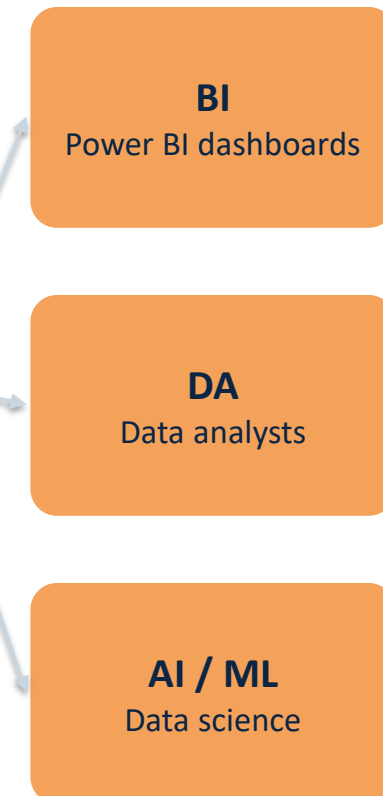
- ADLS holds the raw data
- Databricks + PySpark cleans & arranges it
- Curated tables serve BI, analysts & AI
- ADLS + Databricks + pipelines = the Data Platform

5 Reference architecture – the big picture

SOURCES



CONSUMERS



Blob Storage vs ADLS Gen2

Same account kind – very different behaviour for analytics

06

Flat vs hierarchical namespace

Blob – flat namespace

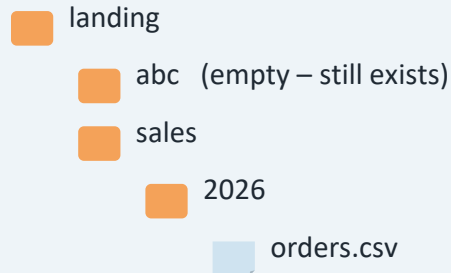
landing/abc/file1.csv

landing/abc/file2.csv

landing/sales/2026/orders.csv

- 'Folders' are only part of the name
- Empty folder disappears
- Rename folder = copy + delete every file
- Analytics slower

ADLS Gen2 – hierarchical



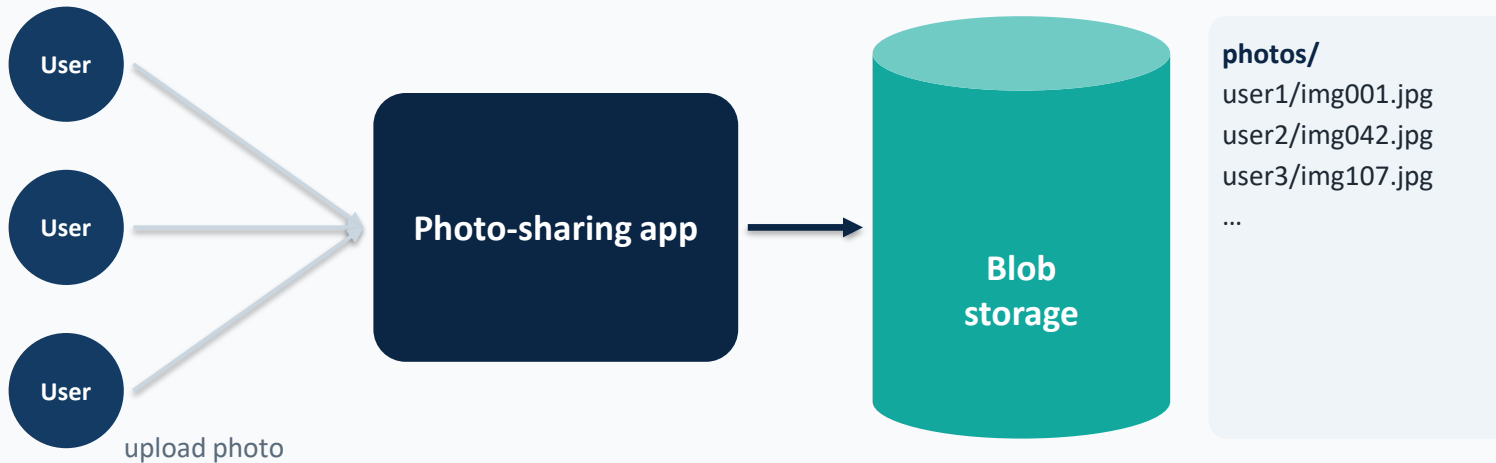
- Real directories
- Rename folder = one instant operation
- ACLs per folder / file
- Faster big-data analytics

Same StorageV2 account – the only switch: Enable hierarchical namespace

KEY POINTS

- Blob: folders are only part of the file name
- Blob: empty folders disappear; renames are slow
- ADLS: real directories, atomic rename
- ADLS: ACLs per folder · faster analytics

When is Blob the right choice?



Blob is ideal for
App images & videos
Documents & backups
Static website files
Log archives

Why not ADLS here?

Each photo is saved and read one at a time by its URL – no big-data jobs scan folders, so a flat namespace is perfect.

KEY POINTS

- Photo-sharing app: users upload photos
- The app stores each photo as a blob
- Read one-by-one by URL – no analytics
- Blob = app files, media, backups, static web

Blob storage vs ADLS Gen2 – full comparison

Feature	Blob (HNS off)	ADLS Gen2 (HNS on)
Account kind	StorageV2	StorageV2 (same)
Namespace	Flat – folders are name prefixes	Hierarchical – real directories
Empty folder	Disappears	Persists
Rename / delete folder	Copy & delete every blob (slow)	Atomic metadata operation (fast)
Security	RBAC, SAS, keys	RBAC + POSIX-style ACLs
Endpoint / driver	blob.core... · wasbs://	dfs.core... · abfss://
SFTP / NFS v3	Not available	Supported
Best for	App files, media, backups	Data lakes & big-data analytics

Lab 2 – Create a Blob Storage Account

...and prove the difference with your own eyes

07

Basics for the Blob account

Subscription * Visual Studio Enterprise Subscription

Resource group * rg-session02-lab
[Create new](#)

Instance details

Storage account name *

Region * (Asia Pacific) South India
[Deploy to an Azure Extended Zone](#)

Preferred storage type Choose preferred storage type

Performance * ☒ Standard: Recommended for most scenarios (general-purpose v2 account)

[Previous](#) [Next](#) [Review + create](#)

KEY POINTS

- Same subscription & resource group rg-session02-lab
- Name: stsession02blob (add your initials)
- Region: same as the ADLS account
- Performance: Standard

Leave hierarchical namespace OFF

Minimum TLS version ⓘ Version 1.2

Permitted scope for copy operations (preview) ⓘ From any storage account

Hierarchical Namespace

Hierarchical namespace, complemented by Data Lake Storage Gen2 endpoint, enables file and directory semantics, accelerates big data analytics workloads, and enables access control lists (ACLs) [Learn more](#)

Enable hierarchical namespace ⓘ ☐

Access protocols

Blob and Data Lake Gen2 endpoints are provisioned by default [Learn more](#)

Enable SFTP ⓘ ☐

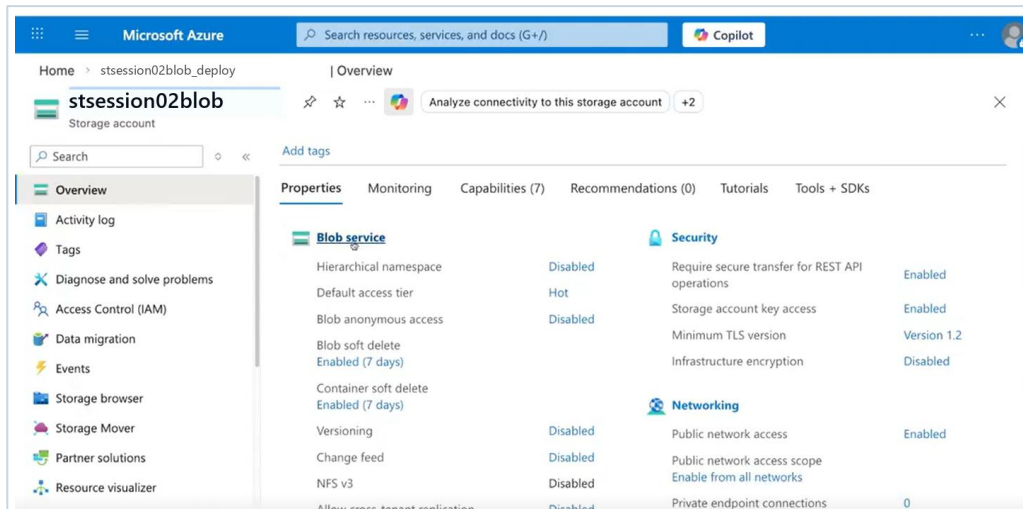
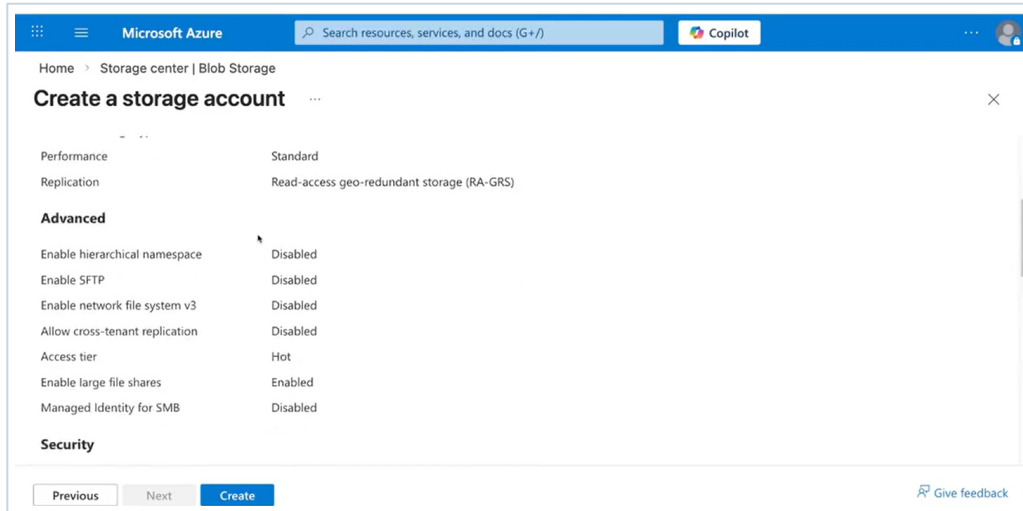
i SFTP can only be enabled for hierarchical namespace accounts

[Previous](#) [Next](#) [Review + create](#)

KEY POINTS

- Enable hierarchical namespace ☐ (unticked)
- ⇒ the account stays plain Blob storage
- SFTP greyed out: 'only for hierarchical namespace accounts'
- A visible Blob vs ADLS difference

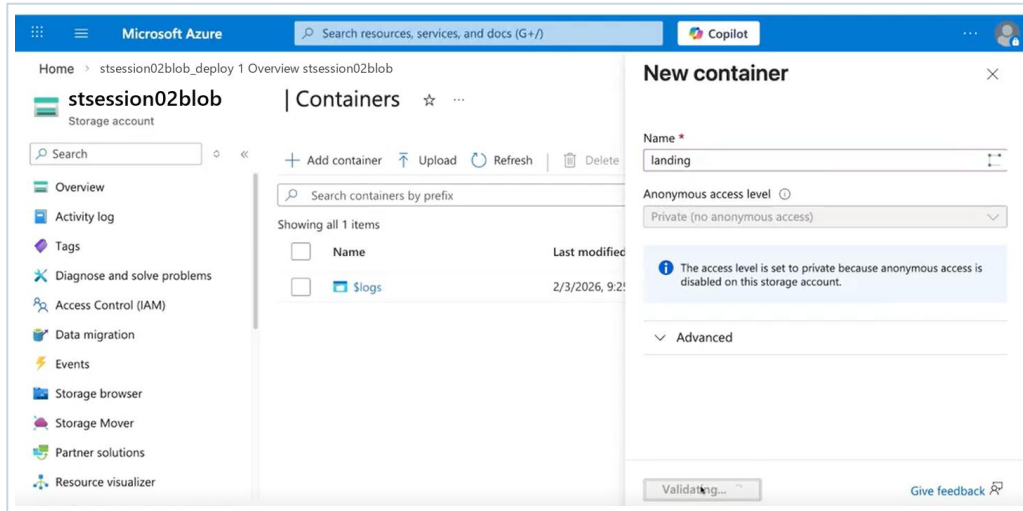
Review, deploy & overview



KEY POINTS

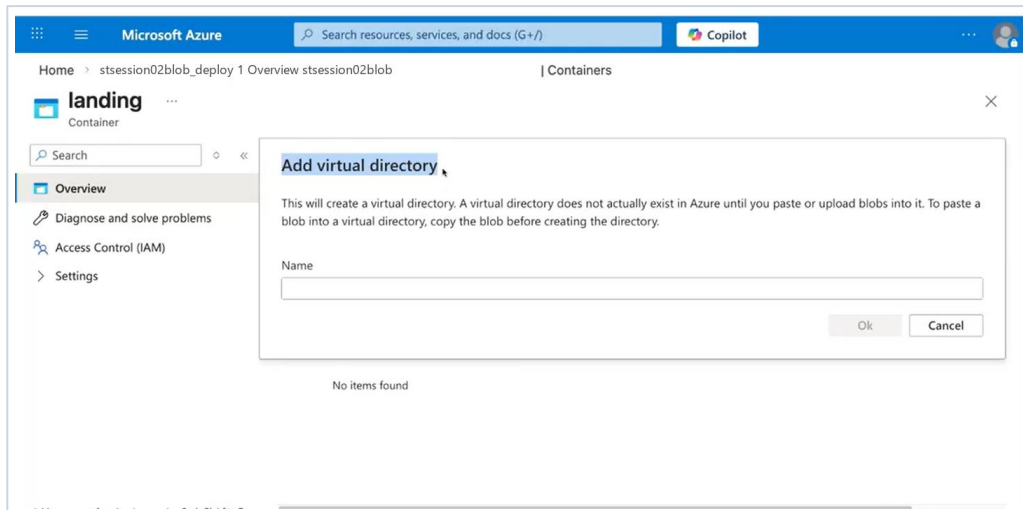
- Review: Hierarchical namespace = Disabled
- Blob & container soft delete: 7 days
- Versioning / change feed: Disabled
- Public network access: all networks

7 Blob container: 'Add virtual directory'



KEY POINTS

- Create container 'landing' (Private)
- 'Add Directory' here creates a VIRTUAL directory
- 'It does not actually exist in Azure ...
- ... until you paste or upload blobs into it'



7 Proof: ADLS vs Blob after the same steps

ADLS Gen2 – folder 'abc' exists (empty)

The screenshot shows the Microsoft Azure portal interface for an ADLS Gen2 container named 'landing'. The breadcrumb navigation is 'Home > stsession02adls | Containers'. The left sidebar shows 'Overview', 'Diagnose and solve problems', 'Access Control (IAM)', and 'Settings'. The main area displays the 'landing' container with an authentication method of 'Access key'. A search bar shows 'Search blobs by prefix (case-sensitive)'. Below the search bar, it says 'Showing all 2 items'. A table lists the items:

Name	Last modified	Access tier	Blob type	Size
abc	2/3/2026, 9:28:05 AM			
sample-fi...	2/3/2026, 8:36:19 AM	Hot (Inferred)	Block blob	48.24 KiB

Blob – empty folder vanished

The screenshot shows the Microsoft Azure portal interface for a Blob container named 'landing'. The breadcrumb navigation is 'Home > stsession02blob_deploy 1 Overview stsession02blob | Containers'. The left sidebar shows 'Overview', 'Diagnose and solve problems', 'Access Control (IAM)', and 'Settings'. The main area displays the 'landing' container with an authentication method of 'Access key'. A search bar shows 'Search blobs by prefix (case-sensitive)'. Below the search bar, it says 'Showing all 1 items'. A table lists the items:

Name	Last modified	Access tier	Blob type	Size
sample-fi...	2/3/2026, 9:29:10 AM	Hot (Inferred)	Block blob	48.24 KiB

Azure Data Factory – Concepts

Azure's cloud ETL & orchestration service

08

What is Azure Data Factory?

Azure Data Factory (ADF)

= ETL jobs

= Data pipelines

MOVE data

Copy from 90+ sources – databases, SaaS apps, files, APIs – into ADLS or a warehouse

ORCHESTRATE

Run Databricks notebooks, SQL, functions in the right order, on a schedule, with retries

Low-code: drag activities onto a canvas & configure them – little or no coding

KEY POINTS

- ADF = ETL jobs = data pipelines
- MOVES data from sources to destinations
- ORCHESTRATES transformations (e.g. Databricks)
- Low-code visual designer

ETL tools in the market

Traditional ETL

(on-premises)

Microsoft SSIS

Informatica

IBM DataStage

Code-based

orchestration

Apache Airflow

(Python DAGs)

Azure-native

✓ our choice

Azure Data Factory

Serverless

Pay per use

90+ connectors

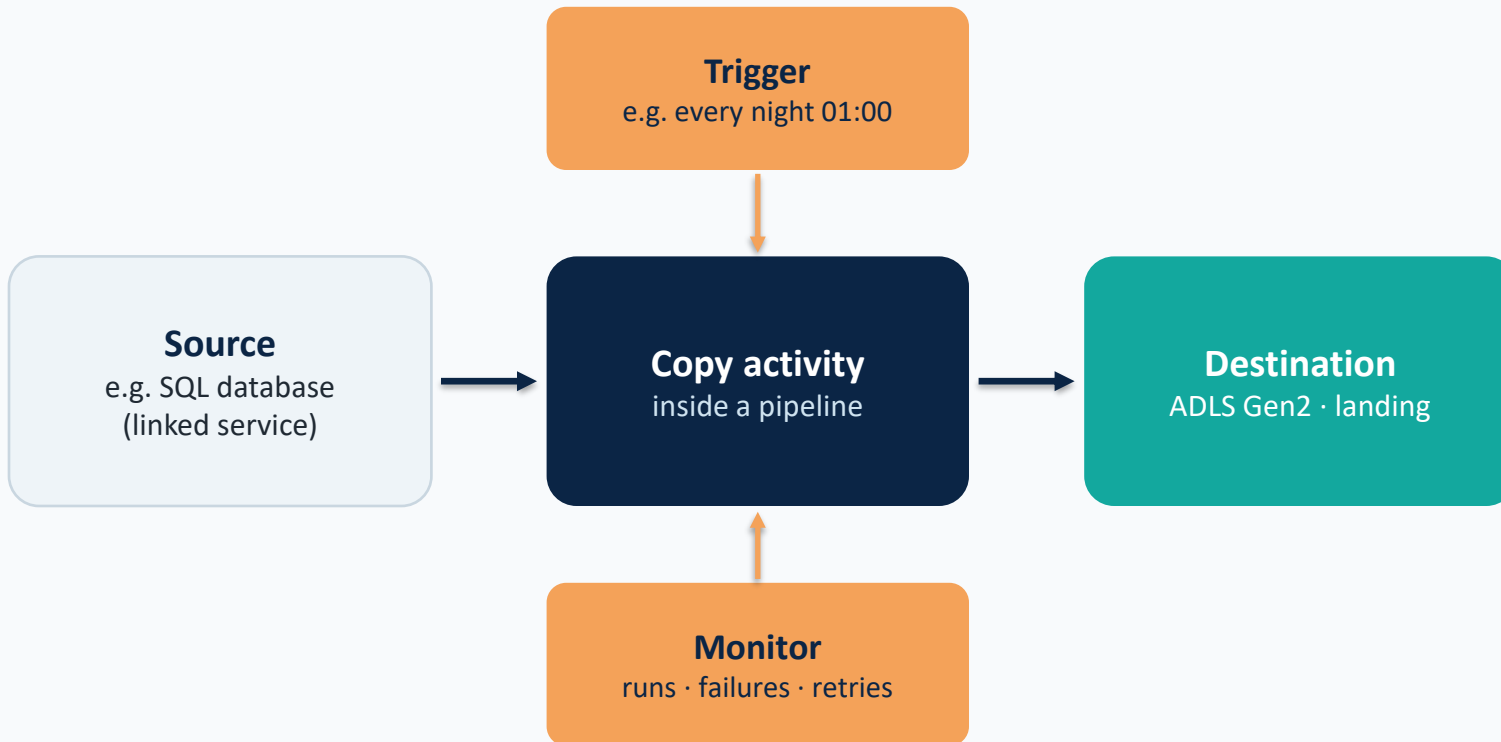
In an Azure project, ADF is the natural choice

Deep integration with ADLS, Databricks, Synapse, Key Vault – and it can run existing SSIS packages

KEY POINTS

- Traditional: SSIS, Informatica, IBM DataStage
- Code-based orchestration: Apache Airflow
- On Azure ⇒ ADF is the native choice
- Serverless, pay-per-use, deep Azure integration

ADF – plug & play



Plug in source → plug in destination → configure → run. No servers to build.

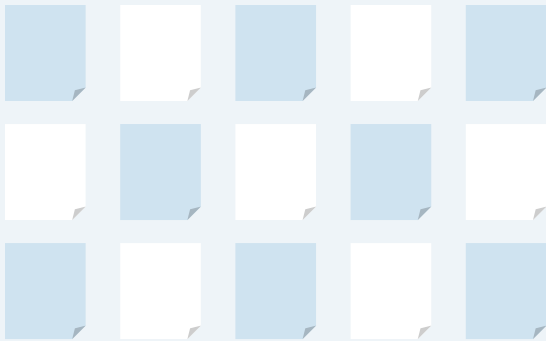
KEY POINTS

- Connect a source (linked service)
- Connect a destination
- Configure a Copy activity
- Trigger it on a schedule, then monitor

Storage vs ADF – where vs how

Storage account

WHERE data lives



1 · 10 · 100 · ... millions of files

Azure Data Factory

HOW & WHEN data moves

pl_ingest_orders

pl_ingest_hr

pl_ingest_crm

pl_transform_sales

... 100s of pipelines

Typically ONE factory per project & environment – holding many pipelines

KEY POINTS

- Storage account holds files: 1 ... millions
- One Data Factory holds many pipelines
- Storage = WHERE data lives
- ADF = HOW and WHEN data moves

Who creates these resources?

Platform / infra / cloud team

creates:

- Subscriptions & resource groups
- Storage accounts
- Data factories & Databricks
- Networking & security

often with IaC: ARM · Bicep · Terraform



Data Engineer

builds inside:

- Linked services & datasets
- Pipelines & triggers
- Transformations (SQL, PySpark)
- Monitoring & fixes

You still learn to create resources: small companies, troubleshooting & interviews

KEY POINTS

- Usually NOT the Data Engineer in big companies
- Platform / infra / cloud teams provision resources
- Often with Infrastructure-as-Code
- Data Engineers build pipelines & transformations inside

Lab 3 – Create an Azure Data Factory

Basics, review, deployment – and what to do when it fails

09

Find the service

The screenshot shows the Microsoft Azure portal interface. At the top, the search bar contains the text 'data facto'. Below the search bar, the results are categorized into 'Services' and 'Resources'. The 'Services' section lists 'Data factories' (highlighted), 'Data factories (V2)', 'Reservations' (with keywords 'Data Factory'), and 'Oracle Database@Azure'. The 'Resources' section lists 'DatabricksDemo' (Azure Databricks Service), 'azuredatafactorystg' (Storage account), and 'azuredatafactorystg-37ec4950-5351-...' (Event Grid System Topic). On the left side of the portal, there is a 'Resources' section with tabs for 'Recent' and 'Favorite'. The 'Recent' tab is active, showing a list of resources with their names and last viewed times.

Name	Last Viewed
stsession02adls	14 minutes
stsession02blob	17 minutes
rg-session02-lab	23 minutes
rg-project-demo	an hour ago

KEY POINTS

- Search 'data factory' in the portal
- Services ► Data factories
- 'Recent' shows the resources created earlier

Data factories list & naming

Home

Data factories

Default Directory (contoso.onmicrosoft.com)

Sort Data factories by creation date. Group Data factories by location in ARG query. +1

+ Create Manage view Refresh Export to CSV Open query Assign tags Add to service group

Filter for any field... Subscription equals all Type equals all Resource Group equals all Location equals all

	Name	Type	Location	Resource Group
☐	adf-demo-dev01	Data factory (V2)	East US	rg-adf-demo
☐	adf-demo-dev02	Data factory (V2)	East US	rg-adf-demo
☐	adf-demo-prod	Data factory (V2)	East US	rg-adf-demo
☐	adf-demo-backup	Data factory (V2)	East US	rg-adf-demo
☐	adf-demo-dev03	Data factory (V2)	East US	rg-adf-demo
☐	adf-demo-uat	Data factory (V2)	East US	rg-adf-demo

KEY POINTS

- All factories are 'Data factory (V2)'
- Suffixes show environments: -dev, -uat, -prod
- Good names make environments obvious
- Click + Create

Create Data Factory – Basics

Microsoft Azure

Search resources, services, and docs (G+)

Copilot

Home > Data factories

Create Data Factory

Resource group * ⓘ

rg-session02-lab

Create new

Instance details

Name * ⓘ

adf-session02-lab

Region * ⓘ

East US

Version * ⓘ

V2

i We recommend you **upgrade to Fabric Data Factory** for a modern, unified data integration experience. [Try Fabric instead!](#)

KEY POINTS

- Tabs: Basics · Git configuration · Networking · Advanced · Tags
- Resource group: rg-session02-lab
- Name: adf-session02-lab (globally unique)
- Region & Version V2
- Banner: Fabric Data Factory = newer SaaS option

Review + create

Basics Git configuration Networking Advanced Tags Review + create

 [View automation template](#)

Basics

Subscription	Visual Studio Enterprise Subscription
Resource group	rg-session02-lab
Name	adf-session02-lab
Region	East US
Version	V2

Previous

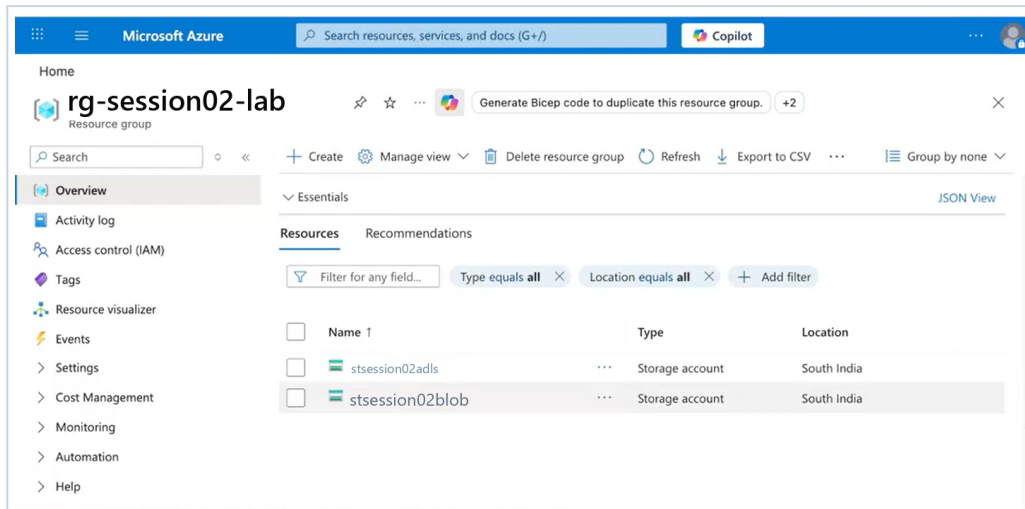
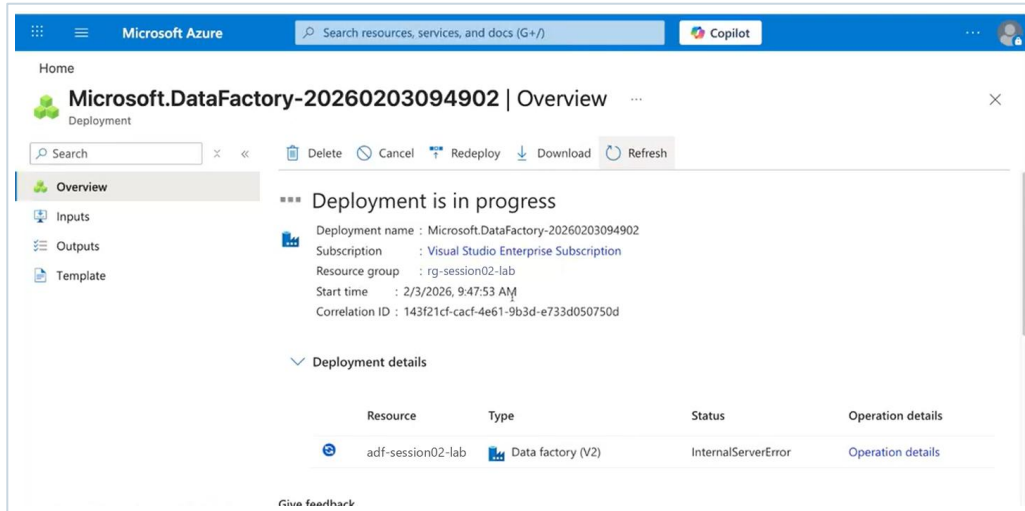
Next

Create

KEY POINTS

- Summary: subscription, RG, name, region, V2
- 'View automation template' again
- Create ⇒ deployment starts

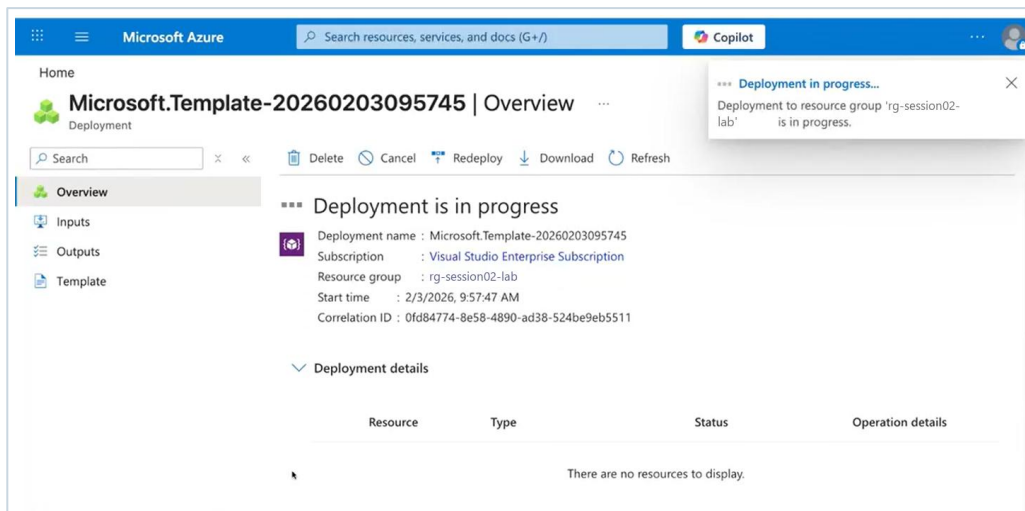
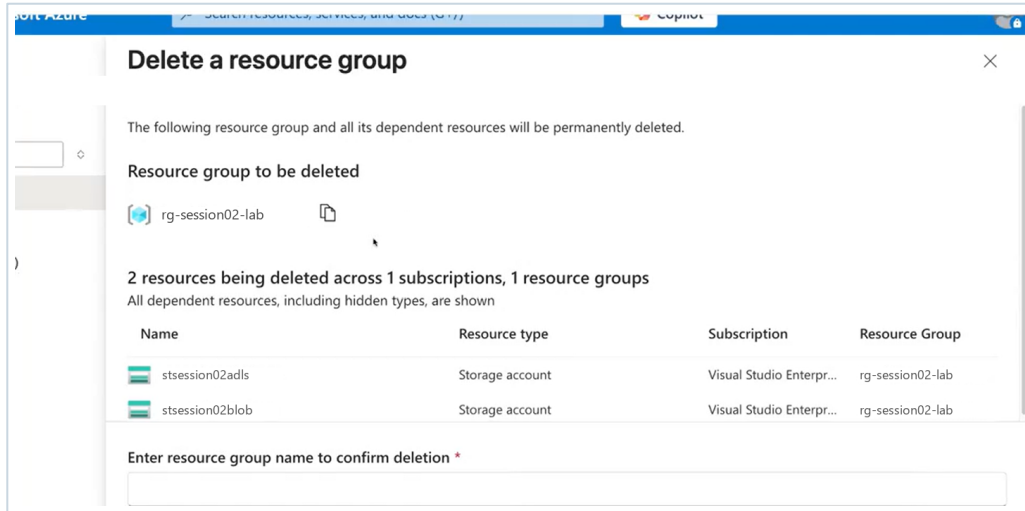
When a deployment fails



KEY POINTS

- Status: InternalServerError (temporary platform error)
- Open 'Operation details' to read the real error
- RG shows only the 2 storage accounts – no factory
- Fix: Redeploy / create again

9 Resource-group clean-up & redeploy



KEY POINTS

- Delete resource group ⇒ ALL resources inside deleted
- Type the RG name to confirm
- Delete lab RGs when a course is finished
- Redeploy ⇒ a new deployment from the template

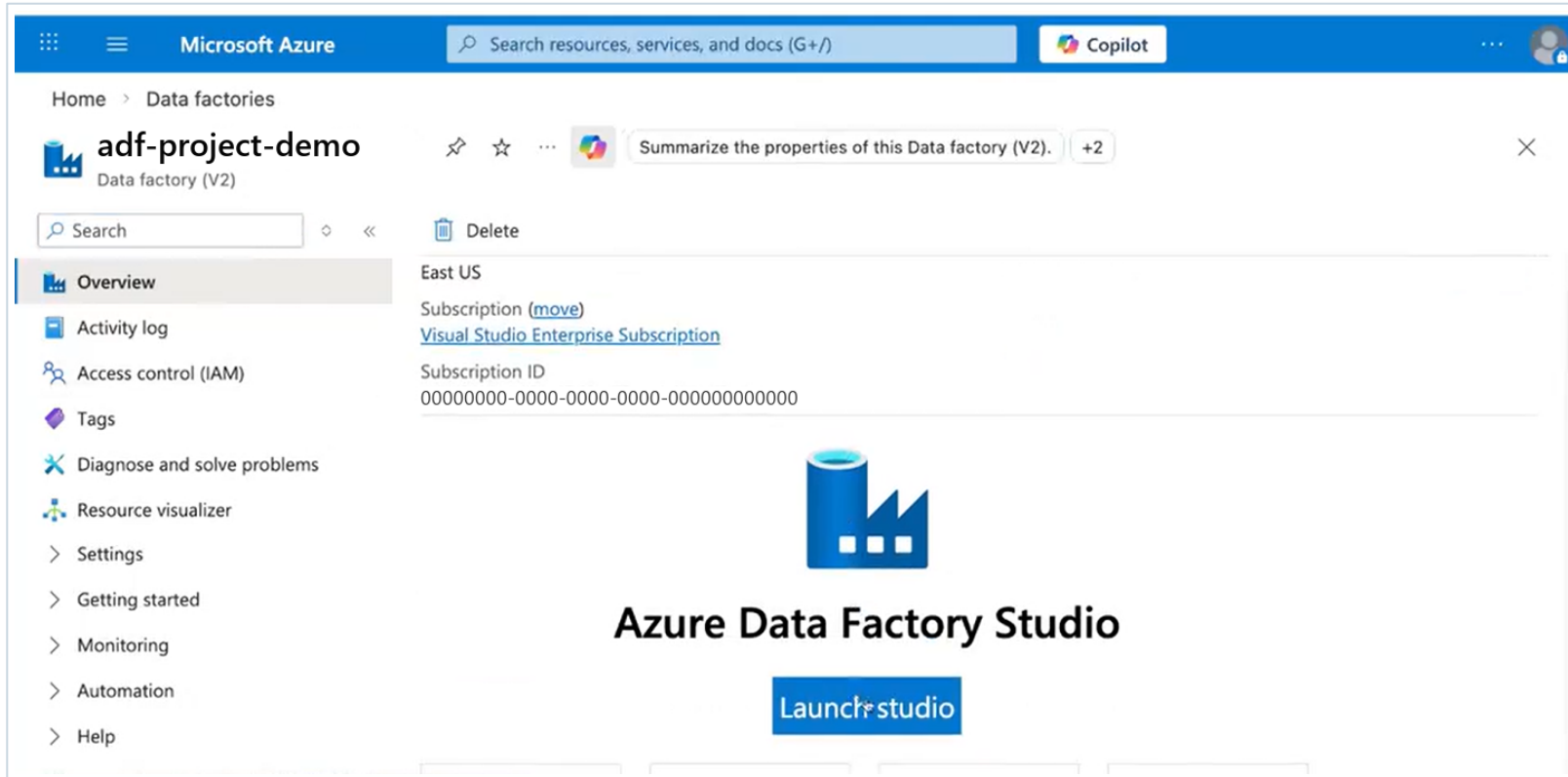
Tour of ADF Studio

Where Data Engineers spend their time



10

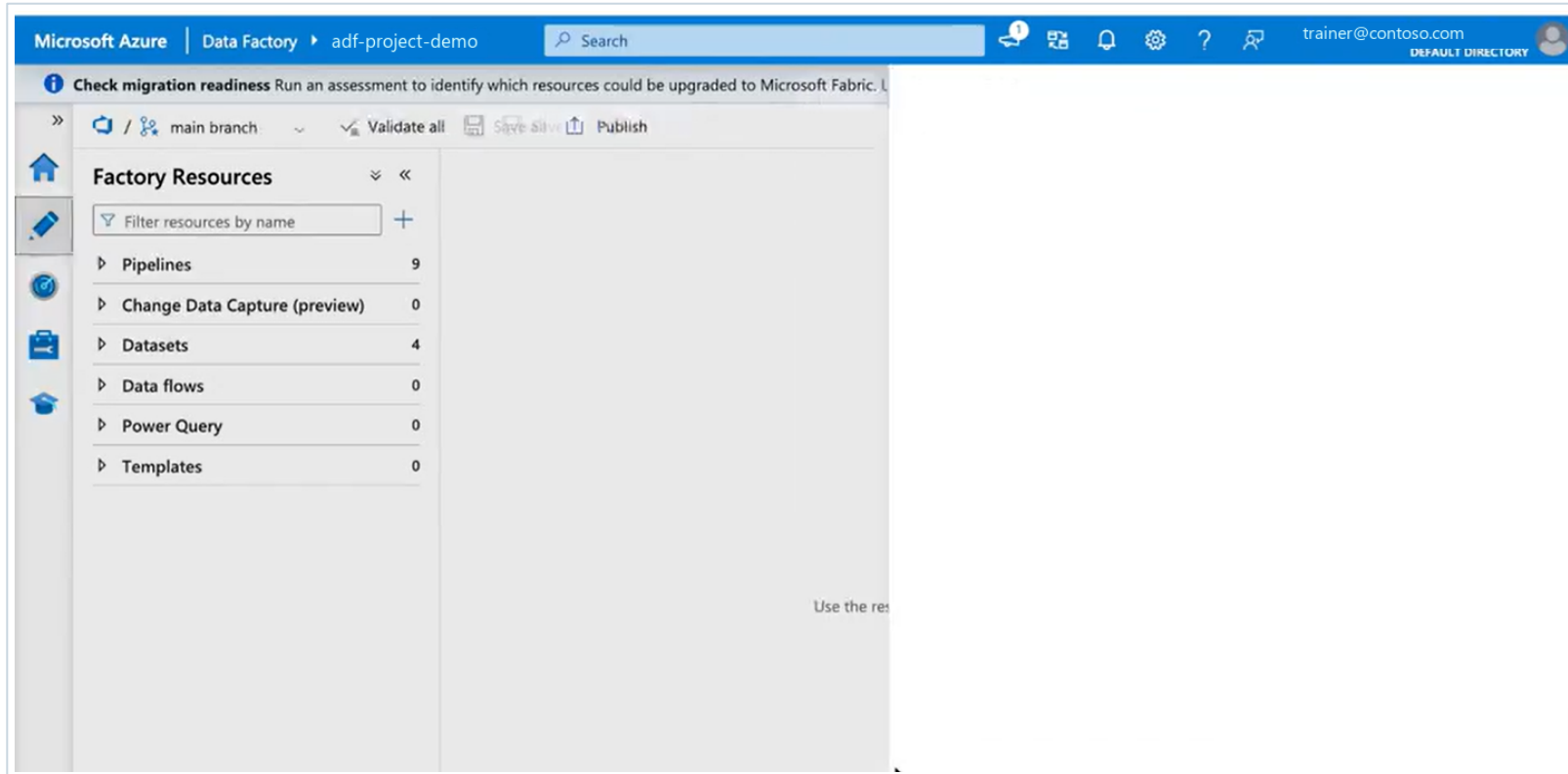
Launch ADF Studio



KEY POINTS

- Factory Overview ► 'Launch studio'
- Opens adf.azure.com
- Portal = manage the resource
- Studio = build, debug, publish, monitor

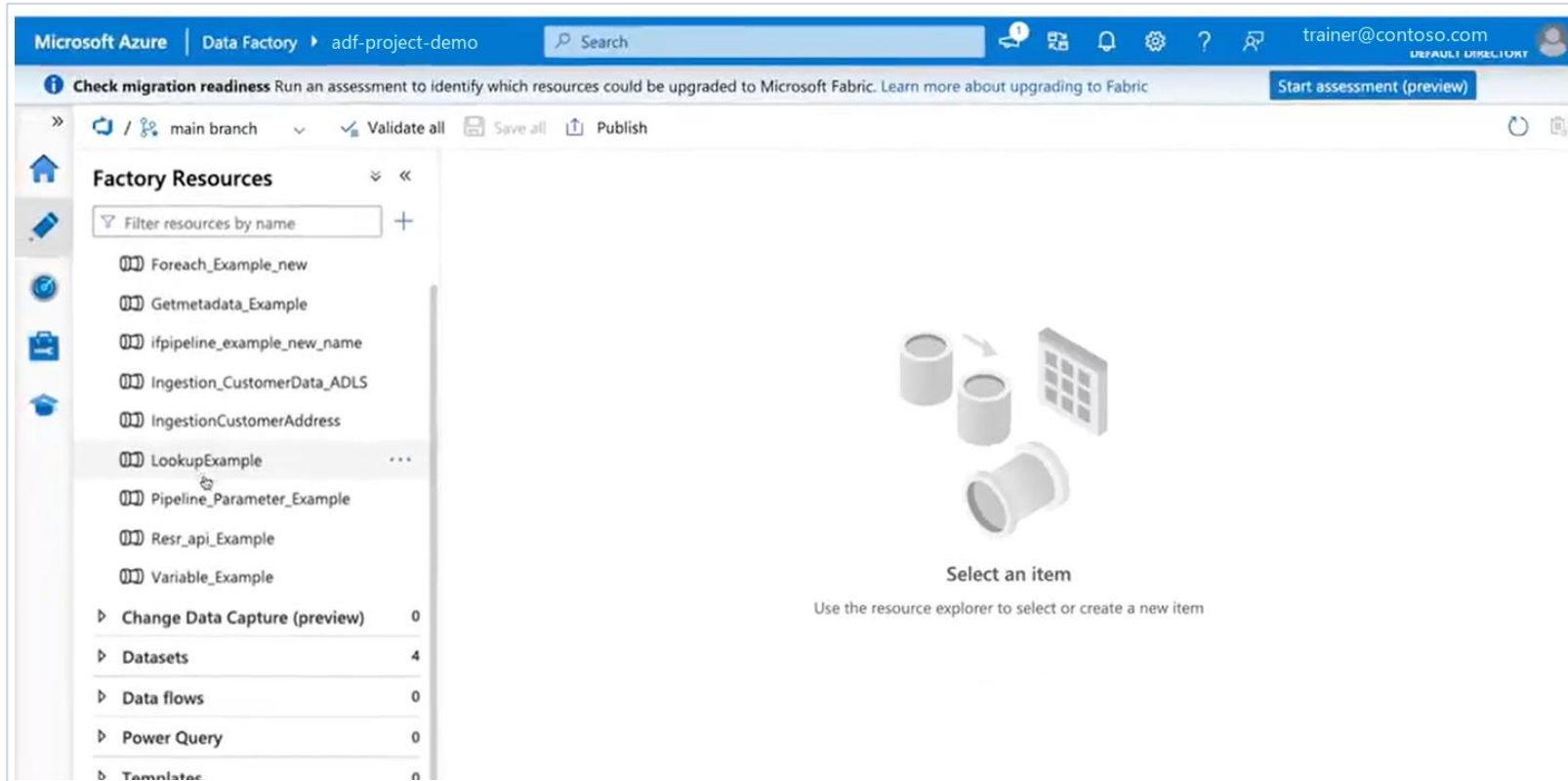
Author hub – Factory Resources



KEY POINTS

- Pipelines · Datasets · Data flows · Power Query · Templates
- Change Data Capture (preview)
- Git branch 'main' · Validate all · Publish
- Hubs: Home · Author · Monitor · Manage · Learning

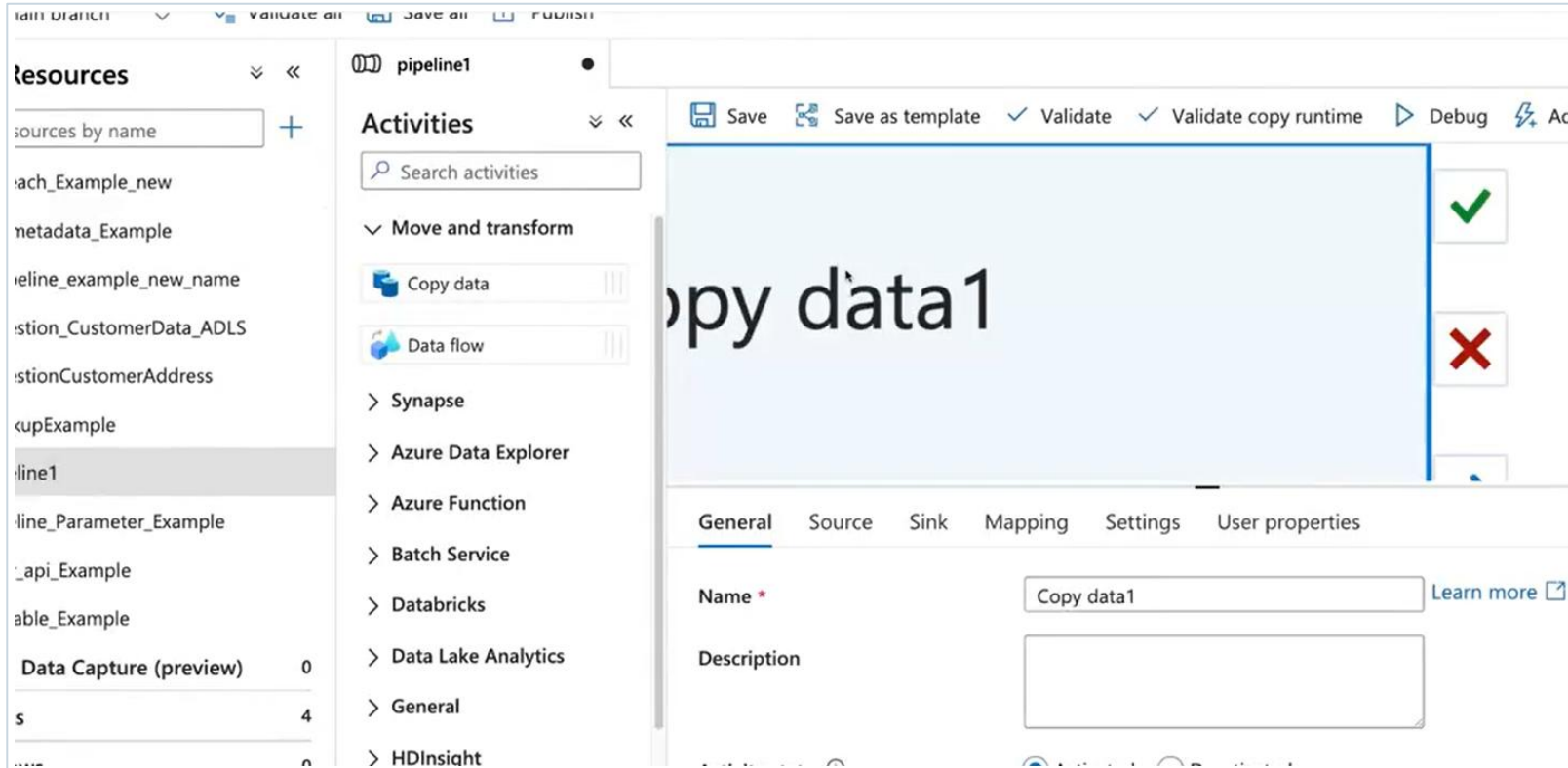
Example pipelines – what's coming next



KEY POINTS

- ForEach, Get Metadata, If Condition, Lookup
- Parameters & variables
- Ingestion into ADLS
- REST API ingestion

Pipeline canvas & activities



KEY POINTS

- Activities: Move and transform
 - Copy data, Data flow
- Also Databricks, Azure Function, HDInsight, General ...
- Activity tabs: General · Source · Sink · Mapping · Settings
- Toolbar: Validate · Debug · Add trigger

Chaining activities

The screenshot shows the Azure Data Factory pipeline editor for a pipeline named 'pipeline1'. The left sidebar displays a list of resources, with 'pipeline1' selected. The main canvas shows a pipeline diagram with three 'Copy data' activities connected sequentially: 'Copy data1' → 'Copy data2' → 'Copy data3'. A green arrow indicates the flow from 'Copy data1' to 'Copy data2'. The bottom panel shows the configuration for the selected 'Copy data1' activity, with tabs for General, Source, Sink, Mapping, Settings, and User properties. The 'General' tab is active, showing fields for Name, Description, Activity state (set to 'Activated'), Timeout (0.12:00:00), and Retry (0).

KEY POINTS

- Copy data1 ⇒ Copy data2 ⇒ Copy data3
- Green arrow = 'on success' dependency
- General: Timeout 12 h · Retry 0 · Interval 30 s
- Add retries for unreliable sources

Copy activity – Source

The screenshot shows the Azure Data Factory (ADF) interface for configuring a Copy activity. The left sidebar displays the 'Resources' pane with a list of datasets, including 'line1'. The 'Activities' pane on the right shows the 'Copy data' activity selected. The main canvas displays a pipeline diagram with three 'Copy data' activities connected sequentially. The 'Source' tab is active, showing the 'Source dataset' dropdown set to 'CSV_ADLS_DS'. Below this, the 'Dataset properties' section is expanded, showing a table with 'Name' and 'Value' columns. The 'fileName' property is highlighted. The 'File path type' section at the bottom shows three radio buttons: 'File path in dataset' (selected), 'Wildcard file path', and 'List of files'.

Resources

- sources by name
- ach_Example_new
- metadata_Example
- eline_example_new_name
- stion_CustomerData_ADLS
- stionCustomerAddress
- cupExample
- line1
- line_Parameter_Example
- _api_Example
- able_Example
- Data Capture (preview) 0
- s 4
- lws 0

Activities

- Search activities
- Move and transform
 - Copy data
- Data flow
- Synapse
- Azure Data Explorer
- Azure Function
- Batch Service
- Databricks
- Data Lake Analytics
- General
- HDInsight

Copy data activity configuration (Source tab):

- Source dataset: CSV_ADLS_DS
- Dataset properties:

Name	Value
fileName	Value
- File path type:
 - ☒ File path in dataset
 - ☐ Wildcard file path
 - ☐ List of files

KEY POINTS

- Source dataset: CSV_ADLS_DS (CSV on ADLS)
- Dataset parameter folderName ⇒ reusable dataset
- File path type: dataset path · wildcard · list of files
- Preview data before running

ADF building blocks

Component	Meaning	Analogy
Pipeline	A workflow – the ETL job	The recipe
Activity	One step (Copy, Lookup, ForEach, Notebook ...)	A step in the recipe
Dataset	WHAT data – file, folder or table	The ingredient
Linked service	WHERE/HOW to connect – connection info	The shop address + key
Integration runtime	The compute that runs activities	The kitchen
Trigger	WHEN – schedule, tumbling window, event	The timer

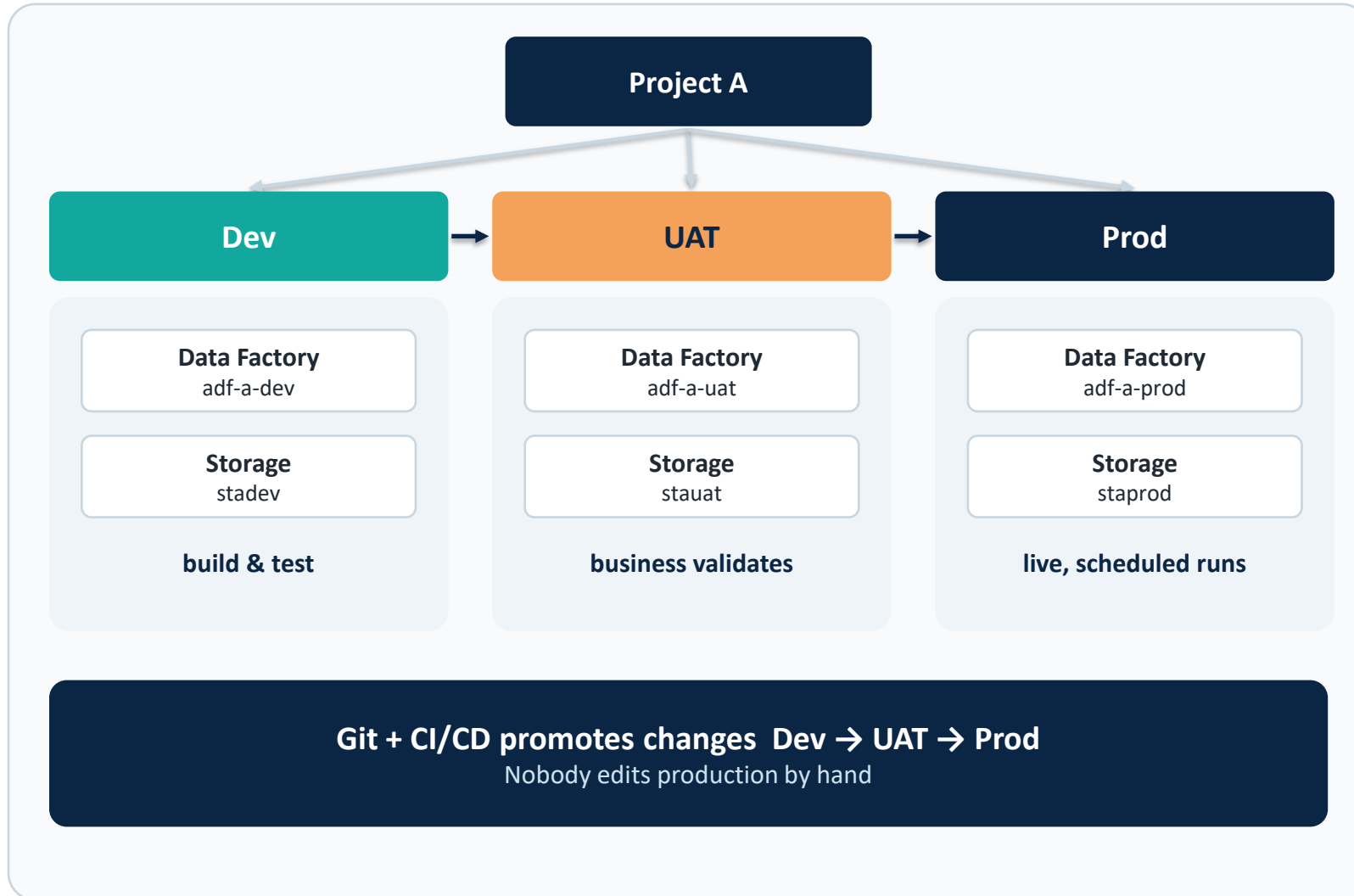
Environments & Summary

Dev · UAT · Prod – and everything learned today



11

Dev / UAT / Prod environments



KEY POINTS

- Each project has Dev, UAT and Prod
- Each environment has its OWN ADF and storage
- Dev: build · UAT: business validates · Prod: live
- Git + CI/CD promotes changes – never edit Prod by hand

Key takeaways

1

Storage Account

Container for Blob, ADLS, Files, Queues, Tables – kind StorageV2.

2

ADLS Gen2

= StorageV2 + hierarchical namespace – real folders, ACLs, fast analytics.

3

Hierarchy

Tenant ▶ Subscription (billing) ▶ Resource group (folder)
▶ Resource; Region = location.

4

OLTP vs OLAP

Apps use OLTP; analytics runs on OLAP; ETL jobs bridge them – the Data Engineer's job.

5

Architecture

Sources ▶ ADF ▶ ADLS ▶ Databricks ▶ BI / analysts / AI.

6

ADF

Pipelines, activities, datasets, linked services, integration runtimes, triggers – built in ADF Studio.

Self-check quiz

1

Q1

Which single setting turns a Blob account into ADLS Gen2?
Can it be undone?

2

Q2

Why does an empty folder disappear in Blob but not in ADLS?

3

Q3

An ATM withdrawal vs a 5-year withdrawals dashboard –
OLTP or OLAP?

4

Q4

Primary South India, Secondary Central India – which
redundancy option?

5

Q5

What happens when you delete a resource group?

6

Q6

Where does the connection information for a source
database live in ADF?

Homework & next steps

1

Repeat the labs

Create ADLS Gen2 + Blob in your own subscription with your own names.

2

Organise the lake

Create containers landing and curated, and folders like landing/sales/2026/.

3

Upload sample data

Upload a small CSV; try Rename on an ADLS folder.

4

Explore ADF Studio

Open the Author, Monitor and Manage hubs; find Linked services.

5

Keep resources

Keep rg-session02-lab – Session 03 copies data from Azure SQL DB into it.

Next: Session 03 – Azure Data Factory implementation

1

ADF end-to-end flow

How linked services, datasets, activities and pipelines fit together.

2

Linked service

The connection – where the data lives and how ADF signs in to it.

3

Dataset

The data itself – a table in the database or a file/folder in ADLS.

4

Copy activity

The step that reads from a source dataset and writes to a sink dataset.

5

Your first cloud database

Create an Azure SQL Database as the source system.

6

Your first ADF pipeline

Move data from Azure SQL DB into ADLS – as CSV and as JSON.



Thank you

Session 02 – Storage Account & Azure Data Factory · Student Notes